

Data Structures and Algorithms with Python

Prof. Sarada Samantaray

LICAI, LNMIIT

1.0 UNION-FIND

- dynamic connectivity
- quick find
- quick union
- Improvements
- applications

Subtext of today's lecture (and this course)

- Steps to developing a usable algorithm.
 - Model the problem.
 - Find an algorithm to solve it.
 - Fast enough? Fits in memory?
 - If not, figure out why.
 - Find a way to address the problem.
 - Iterate until satisfied.
- The scientific method.
- Mathematical analysis.

1.5 UNION-FIND

- **dynamic connectivity**

- quick find
- quick union
- Improvements
- applications

Dynamic connectivity

Given a set of N objects.

- Union command: connect two objects.
- Find/connected query: is there a path connecting the two objects?

union (4,3)

union (3,8)

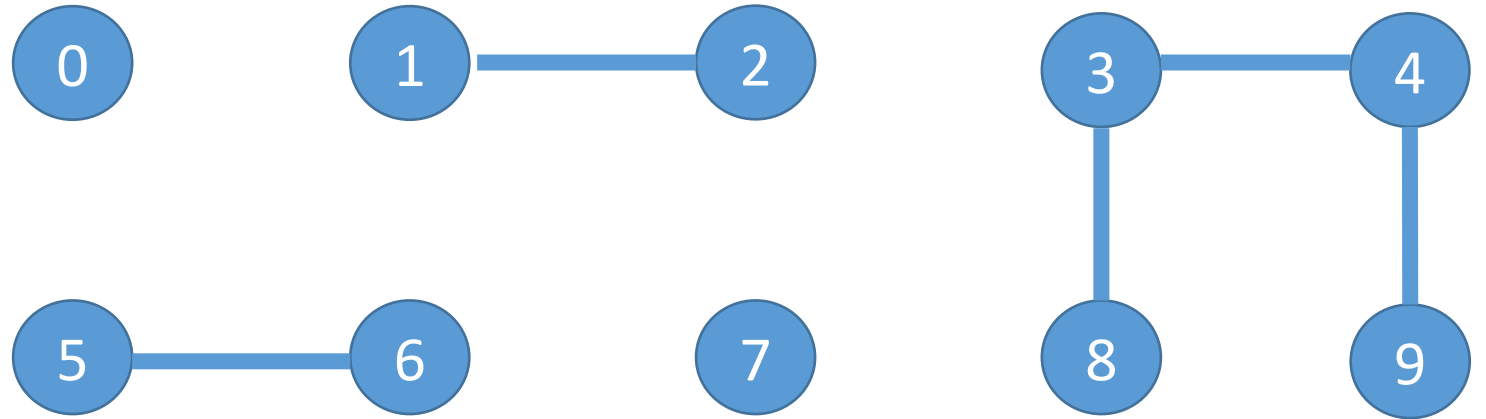
union (6,5)

union (9,4)

union (2,1)

connected (0,7) ✗

connected (8,9) ✓



union (4,3)

union (3,8)

union (6,5)

union (9,4)

union (2,1)

connected (0,7) ✗

connected (8,9) ✓

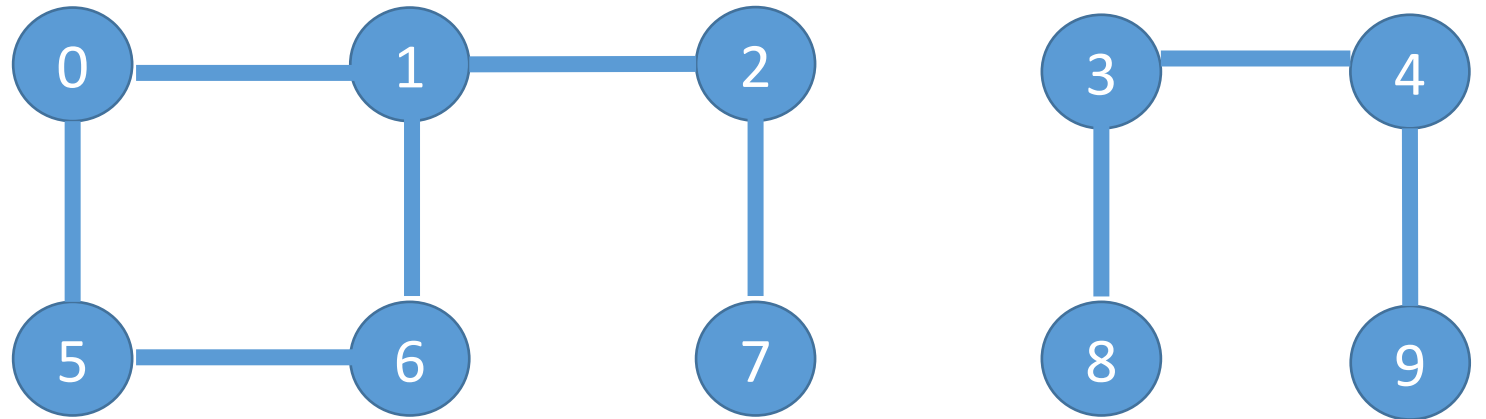
union (5,0)

union (7,2)

union (6,1)

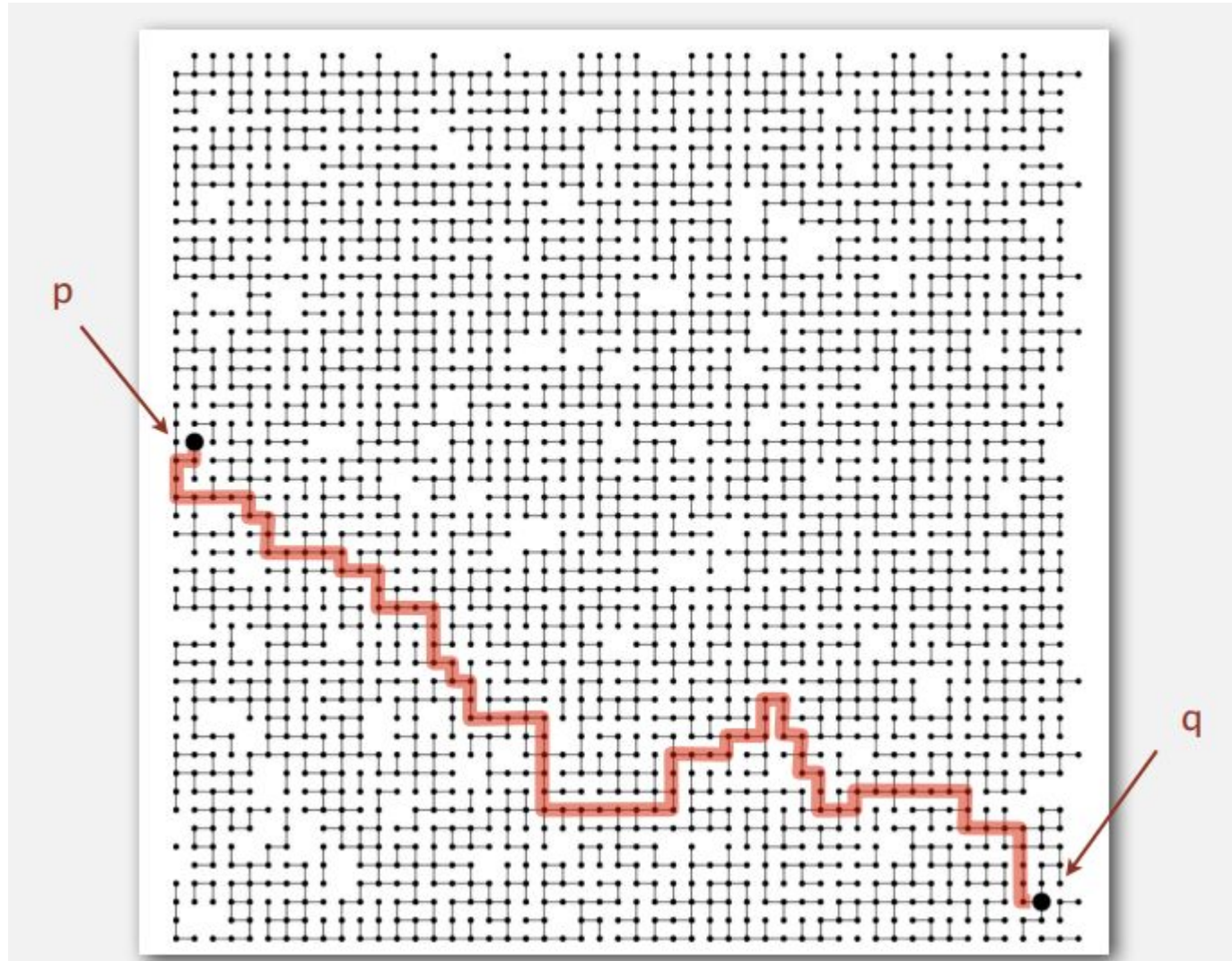
union (1,0)

connected (0,7) ✓



Connectivity example

- Q. Is there a path connecting p and q ?



- Ans: Yes

Modelling the objects

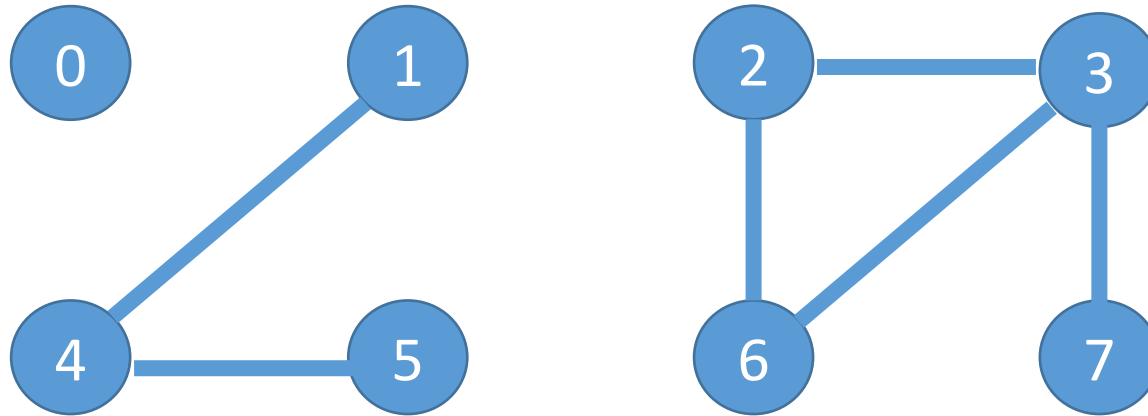
- Applications involve manipulating objects of all types.
 - Pixels in a digital photo.
 - Computers in a network.
 - Friends in a social network.
 - Transistors in a computer chip.
 - Elements in a mathematical set.
 - Variable names in Fortran program.
 - Metallic sites in a composite system.
 - When programming, convenient to name objects 0 to $N - 1$.
 - Use integers as array index.
 - Suppress details not relevant to union-find.
-  can use symbol table to translate from site names to integers: stay tuned (Chapter 3)

Modelling the connections

- We assume "is connected to" is an equivalence relation:
 - **Reflexive**: p is connected to p.
 - **Symmetric**: if p is connected to q, then q is connected to p.
 - **Transitive**: if p is connected to q and q is connected to r, then p is connected to r.
- Connected components. Maximal set of objects that are mutually connected.

{0} {1 4 5} {2 3 6 7}

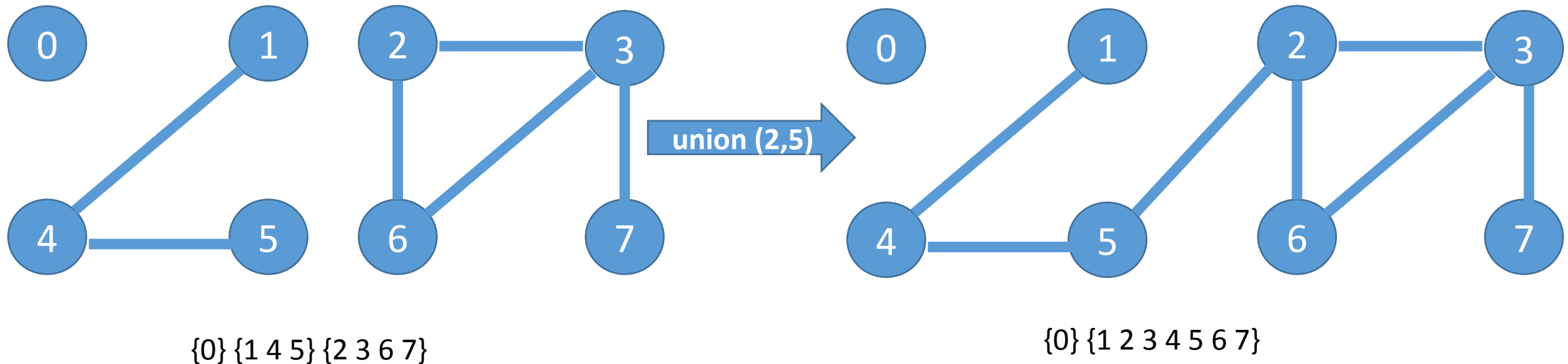
3 connected components



Implementing the operations

Find query. Check if two objects are in the same component.

Union command. Replace components containing two objects with their union.



Union-find data type (API)

Goal. Design efficient data structure for union-find.

- Number of objects N can be huge.
- Number of operations M can be huge.
- Find queries and union commands may be intermixed.

```
class Inmai_uf
```

```
    def __init__(self, n): pass      initialize union-find data structure  
        with  $N$  objects (0 to  $N - 1$ )  
    def union(self, p, q): pass      add connection between  $p$  and  $q$   
    def connected(self, p, q): pass  are  $p$  and  $q$  in the same component?  
    def find(self, p): pass          component identifier for  $p$  (0 to  $N - 1$ )  
    def count(self): pass            number of components
```

Dynamic-connectivity client

- Read in number of objects N from standard input.
- Repeat:
 - read in pair of integers from standard input
 - if they are not yet connected, connect them and print out pair

with open('tinyuf.txt') as hfile:

```
no_of_elm = int(hfile.readline())
```

```
uf = Inmaif_uf(no_of_elm)
```

```
print('number of element in uf: ', uf.count())
```

```
for line in hfile:
```

```
    p, q = line.strip().split()
```

```
    if !uf.connected(p, q):
```

```
        uf.union(p, q)
```

```
        print('connecting - p: ', p, 'q: ', q)
```

% more
tinyUF.txt

10

4 3

3 8

6 5

9 4

2 1

8 9

5 0

7 2

6 1

1 0

6 7

QUIZ

How many connected components result after performing the following sequence of *union* operations on a set of 10 items?

1–2 3–4 5–6 7–8 7–9 2–8 0–5 1–9

- a) 1
- b) 2
- c) 3
- d) 4

1.5 UNION-FIND

- **dynamic connectivity**

- quick find
- quick union
- Improvements
- applications

1.5 UNION-FIND

- dynamic connectivity

- **quick find**

- quick union
- Improvements
- applications

Quick-find [eager approach]

Data structure.

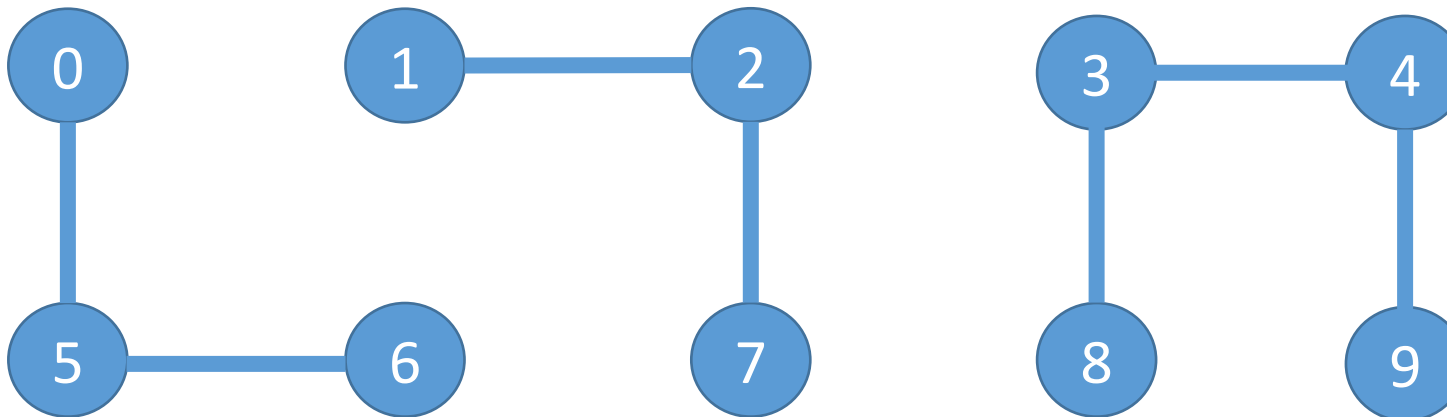
- Integer array `cid[]` of length `N`.
- Interpretation: `p` and `q` are connected iff they have the same id.

if and only if



	0	1	2	3	4	5	6	7	8	9
<code>cid[]</code>	0	1	1	8	8	0	0	1	8	8

0, 5 and 6 are connected
1, 2, and 7 are connected
3, 4, 8, and 9 are connected



Quick-find [eager approach]

Data structure.

- Integer array `cid[]` of length `N`.
- Interpretation: `p` and `q` are connected iff they have the same id.

	0	1	2	3	4	5	6	7	8	9
<code>cid[]</code>	0	1	1	8	8	0	0	1	8	8

connected (6, 1)

Find. Check if `p` (6) and `q` (1) have the same id.

`cid[6] = 0; cid[1] = 1`
6 and 1 are not connected

Quick-find [eager approach]

Data structure.

- Integer array `cid[]` of length `N`.
- Interpretation: `p` and `q` are connected iff they have the same id.

	0	1	2	3	4	5	6	7	8	9
<code>cid[]</code>	0	1	1	8	8	0	0	1	8	8

union (6, 1)

Union. To merge components containing `p` and `q`, change all entries whose id equals `cid[p]` to `cid[q]`.

after union of 6 and 1

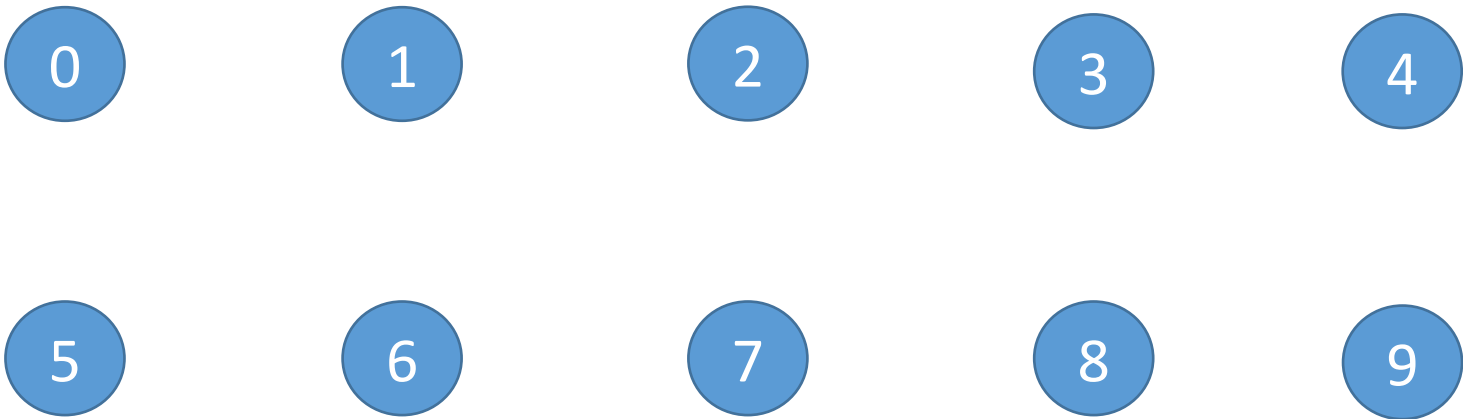
	0	1	2	3	4	5	6	7	8	9
<code>cid[]</code>	1	1	1	8	8	1	1	1	8	8



problem: many values can change

Quick demo

union (4, 3)

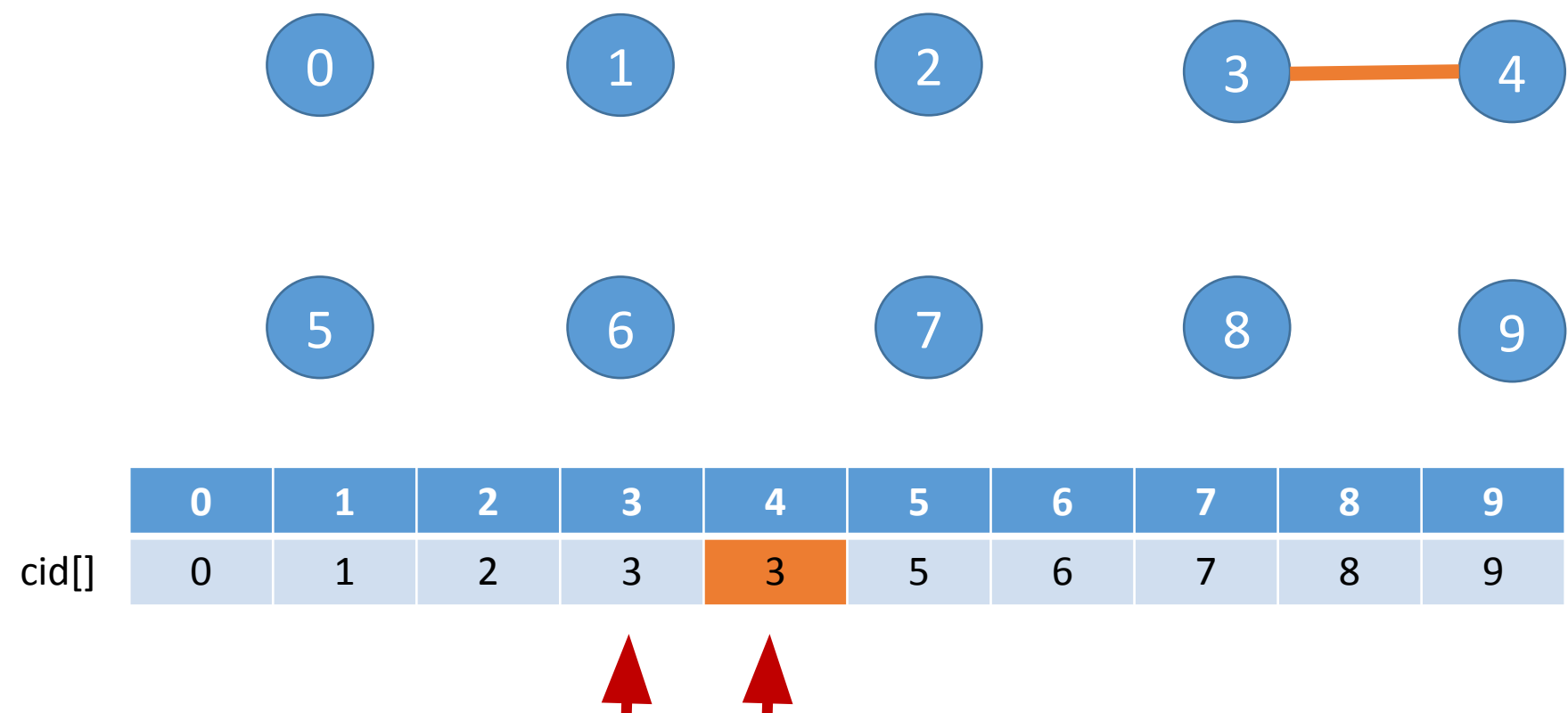


	0	1	2	3	4	5	6	7	8	9
cid[]	0	1	2	3	4	5	6	7	8	9



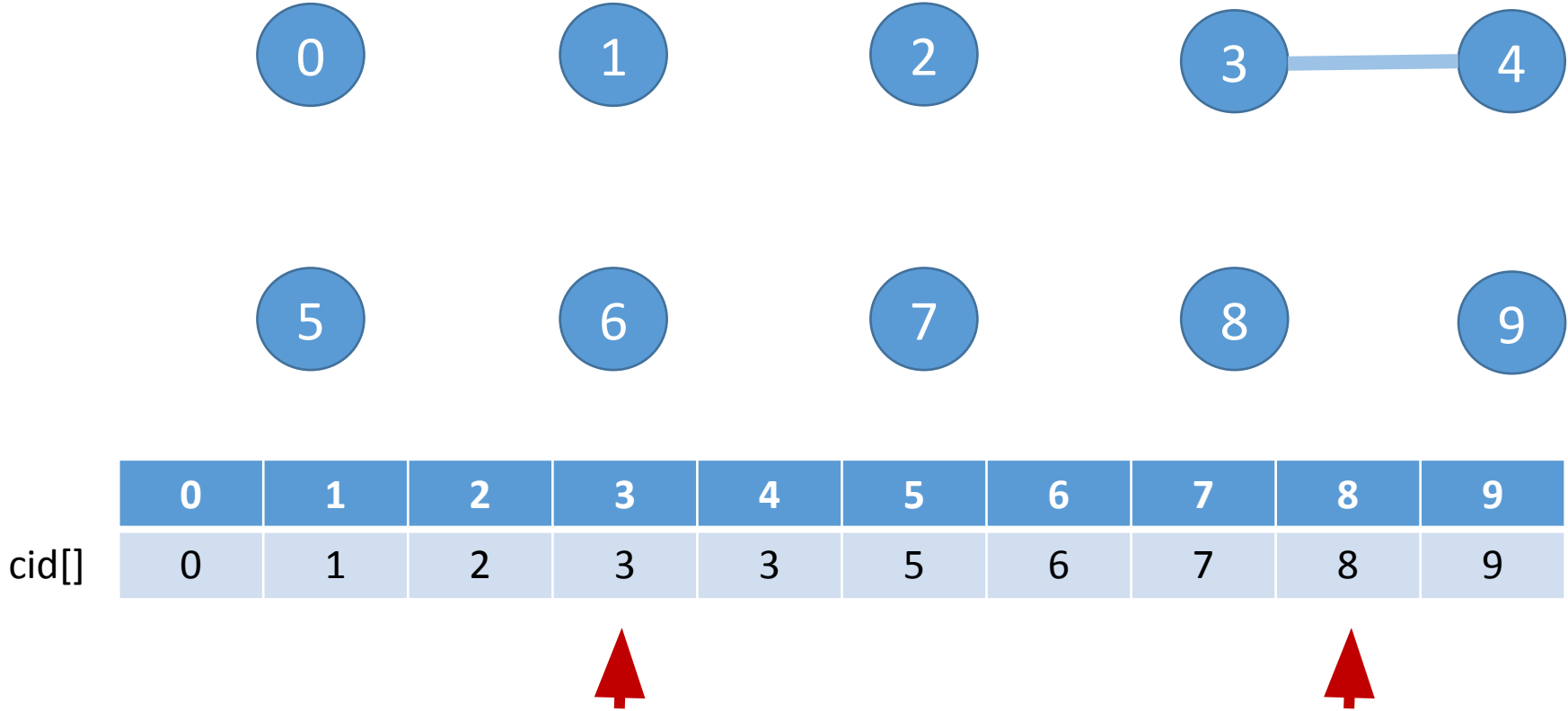
Quick demo

union (4, 3)



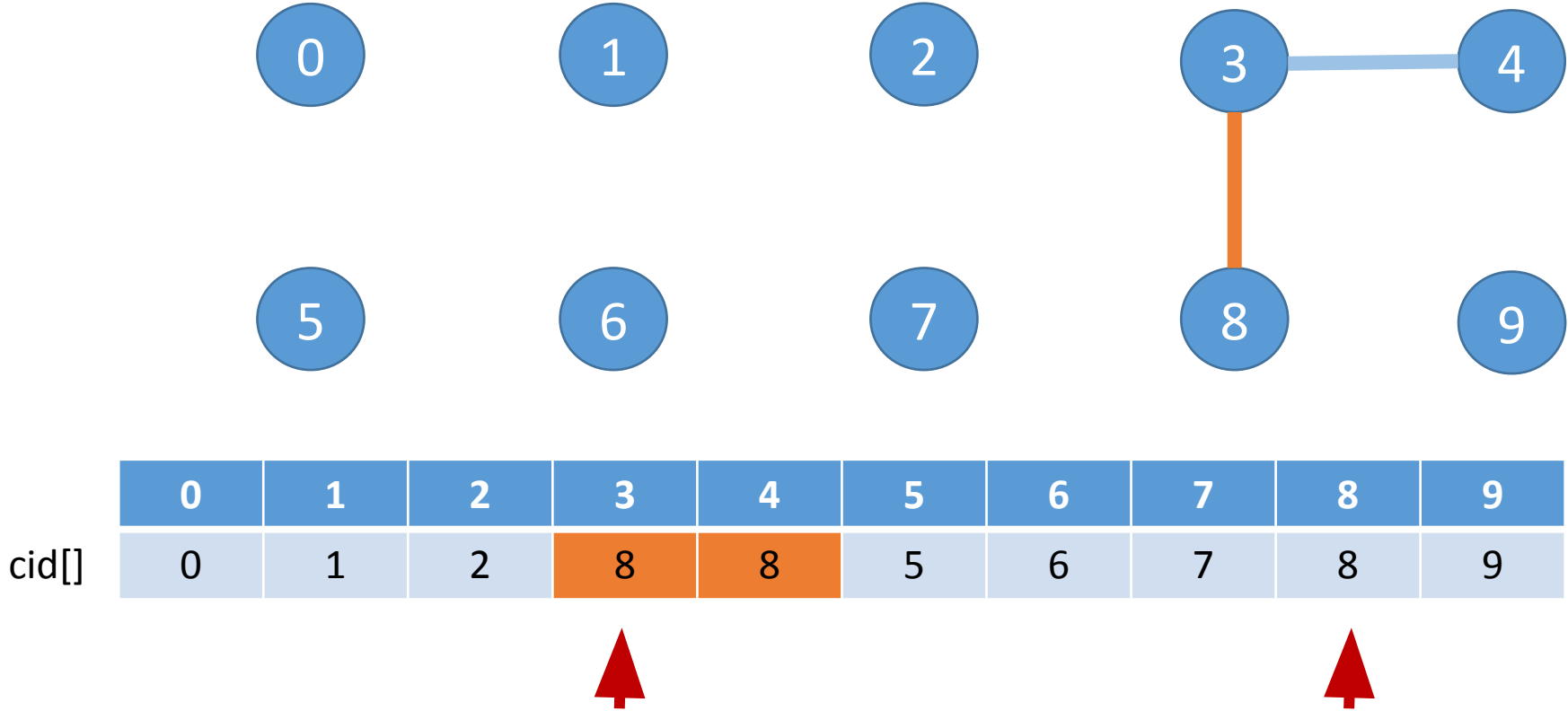
Quick demo

union (3, 8)



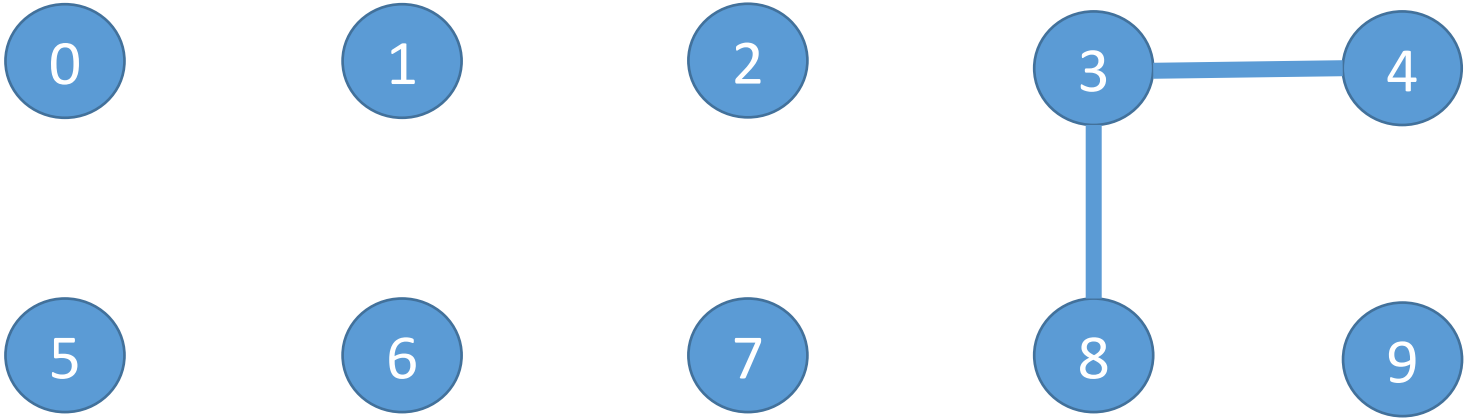
Quick demo

union (3, 8)



Quick demo

union (6, 5)

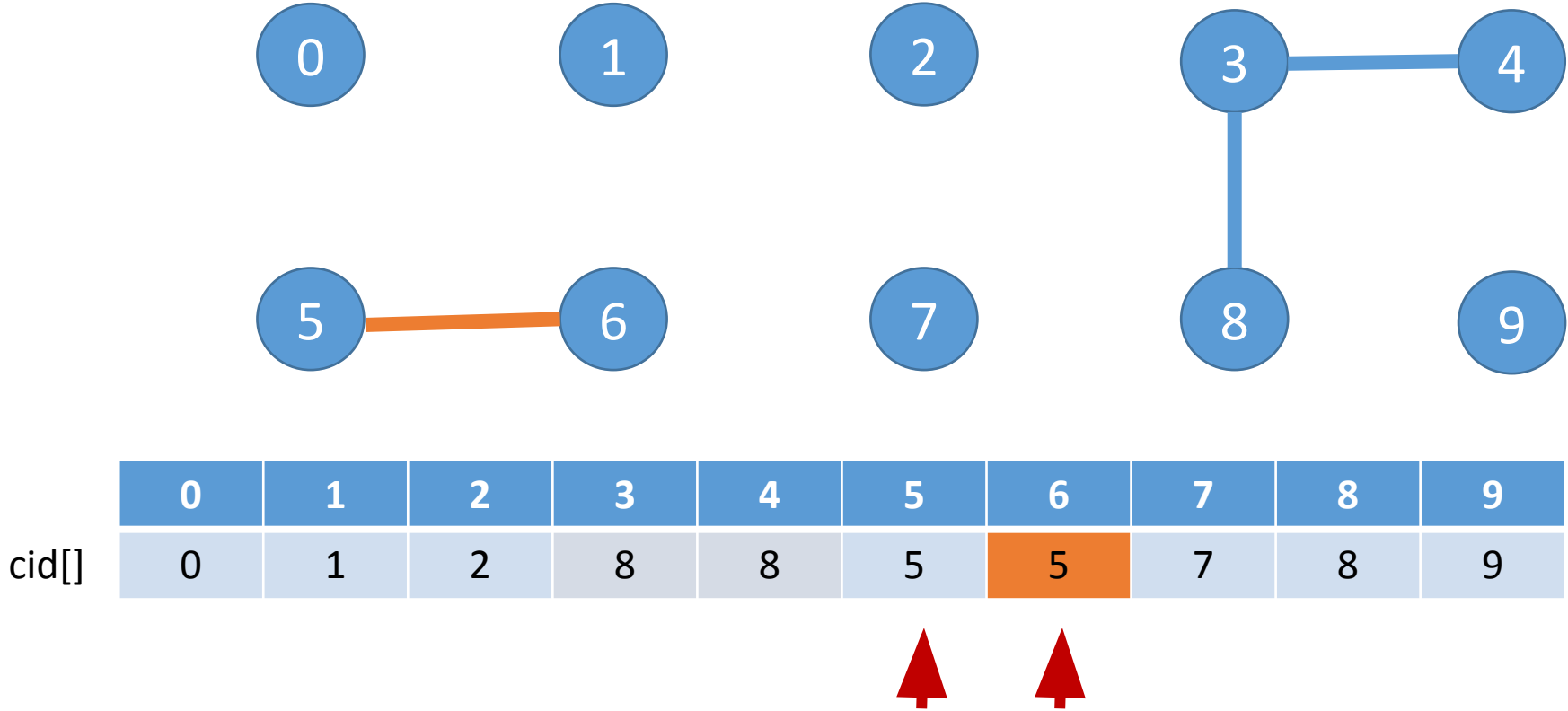


	0	1	2	3	4	5	6	7	8	9
cid[]	0	1	2	8	8	5	6	7	8	9



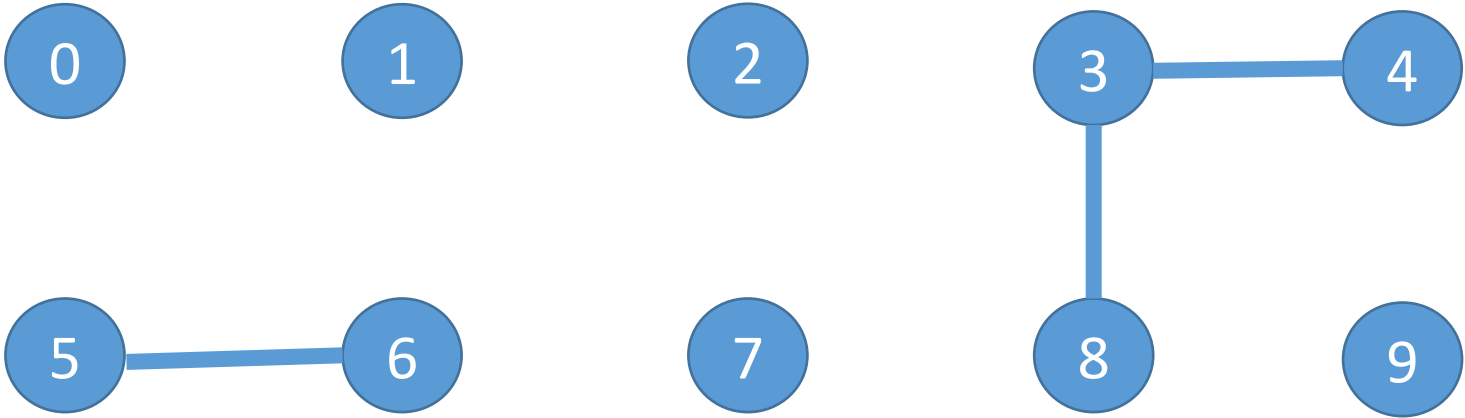
Quick demo

union (6, 5)



Quick demo

union (9, 4)

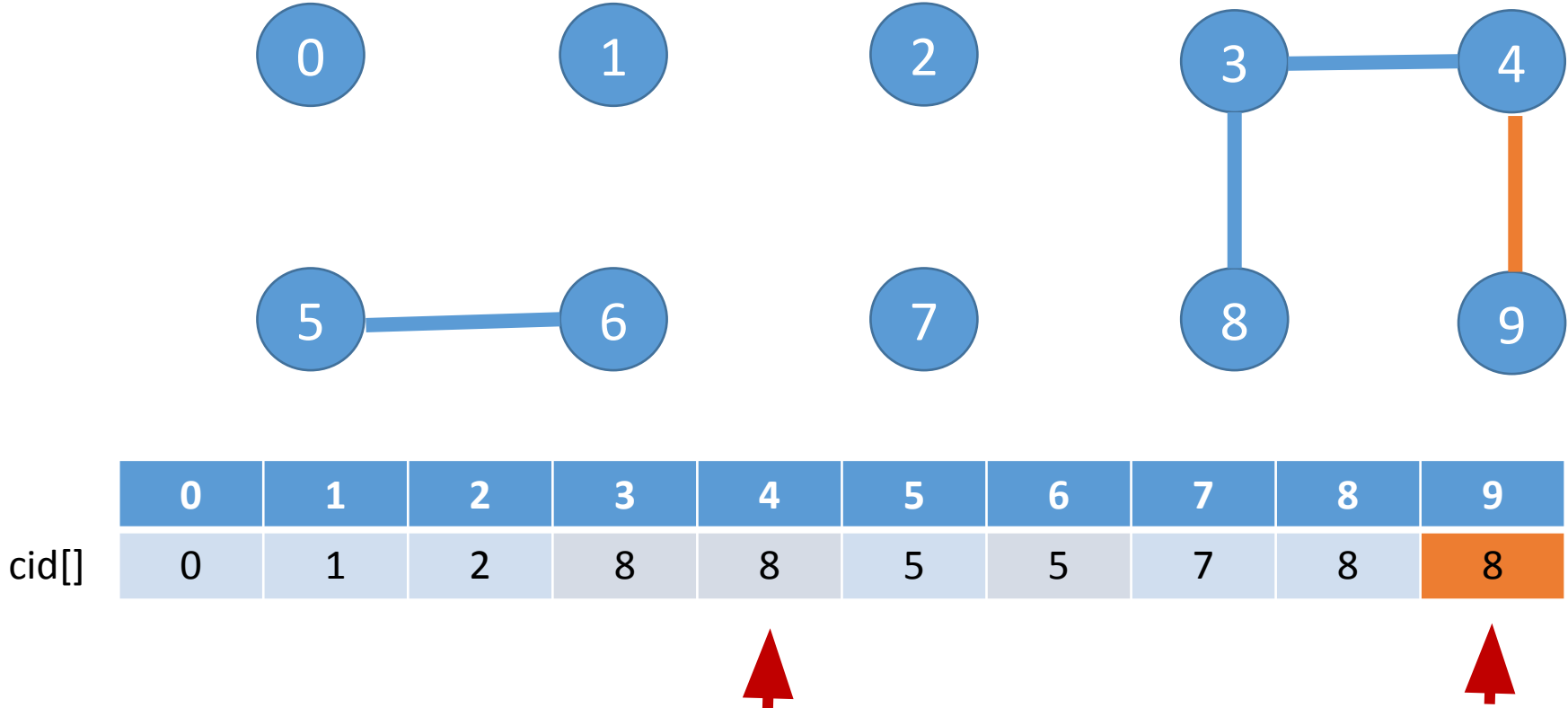


	0	1	2	3	4	5	6	7	8	9
cid[]	0	1	2	8	8	5	5	7	8	9



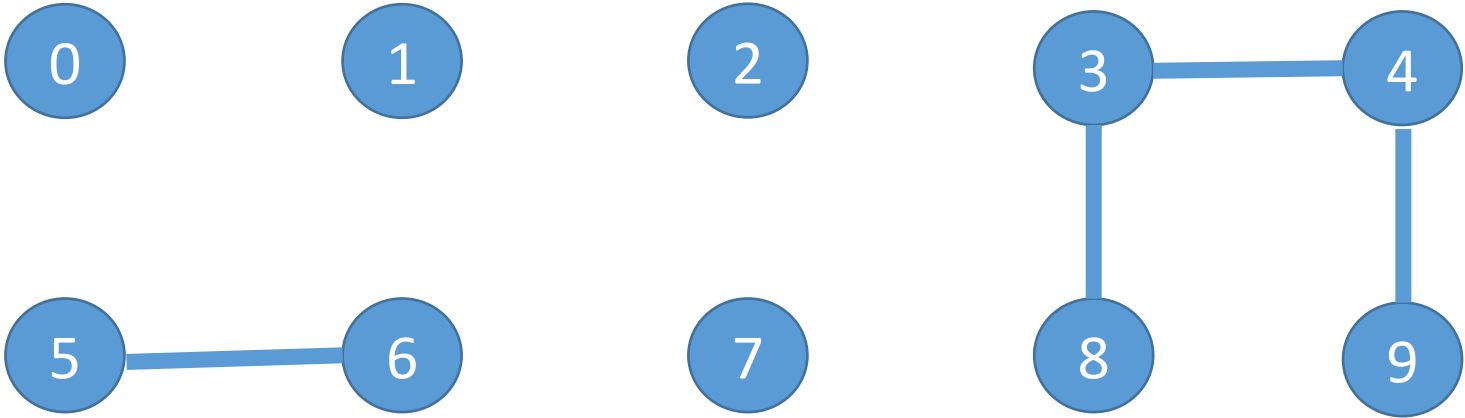
Quick demo

union (9, 4)



Quick demo

union (2, 1)

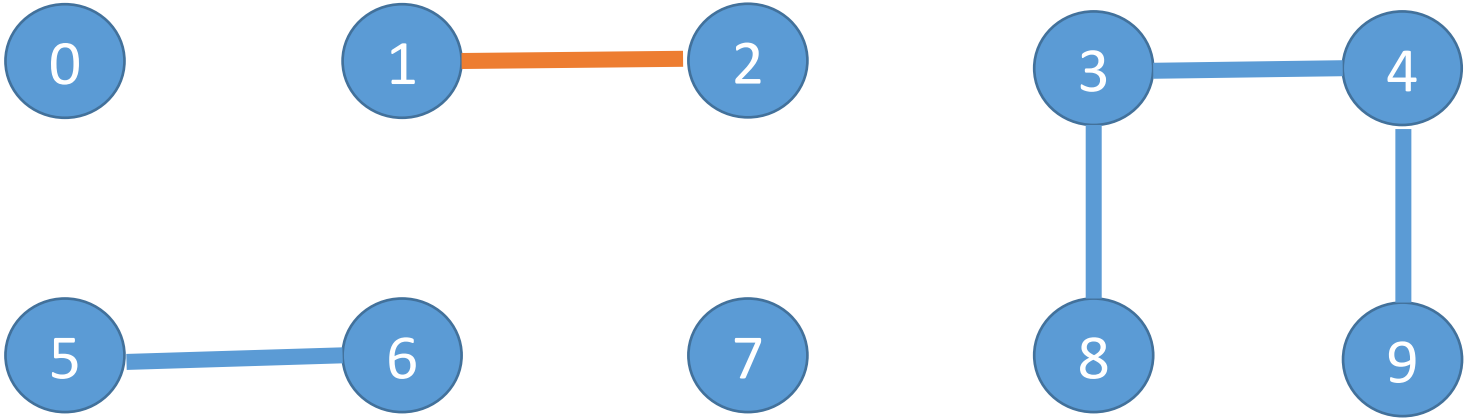


	0	1	2	3	4	5	6	7	8	9
cid[]	0	1	2	8	8	5	5	7	8	8

Two red arrows point to the values 1 and 2 in the cid[] array, corresponding to indices 1 and 2.

Quick demo

union (2, 1)

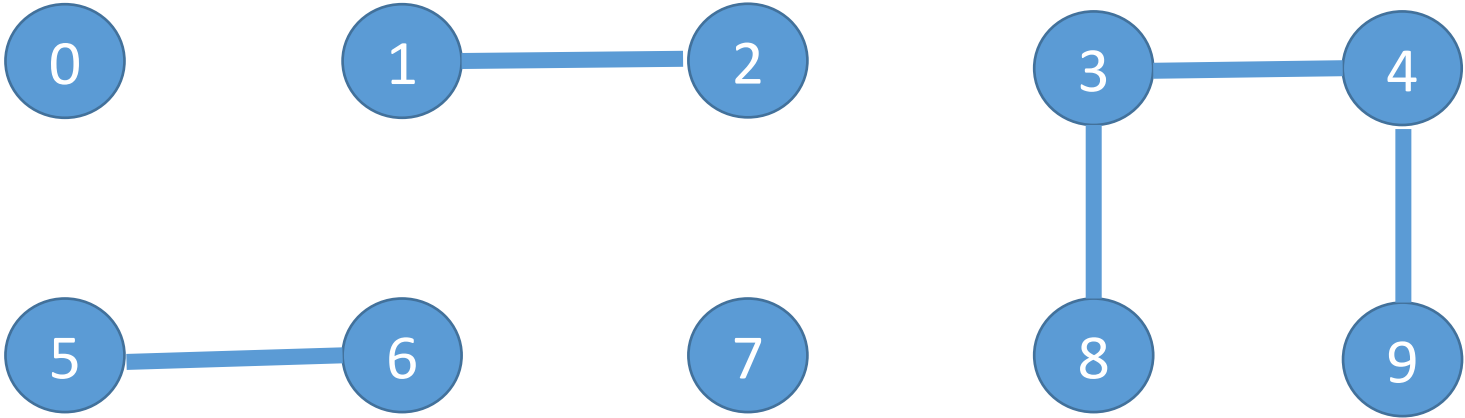


	0	1	2	3	4	5	6	7	8	9
cid[]	0	1	1	8	8	5	5	7	8	8

Two red arrows point to the values 1 in the cid[] array at indices 1 and 2.

Quick demo

union (8, 9)



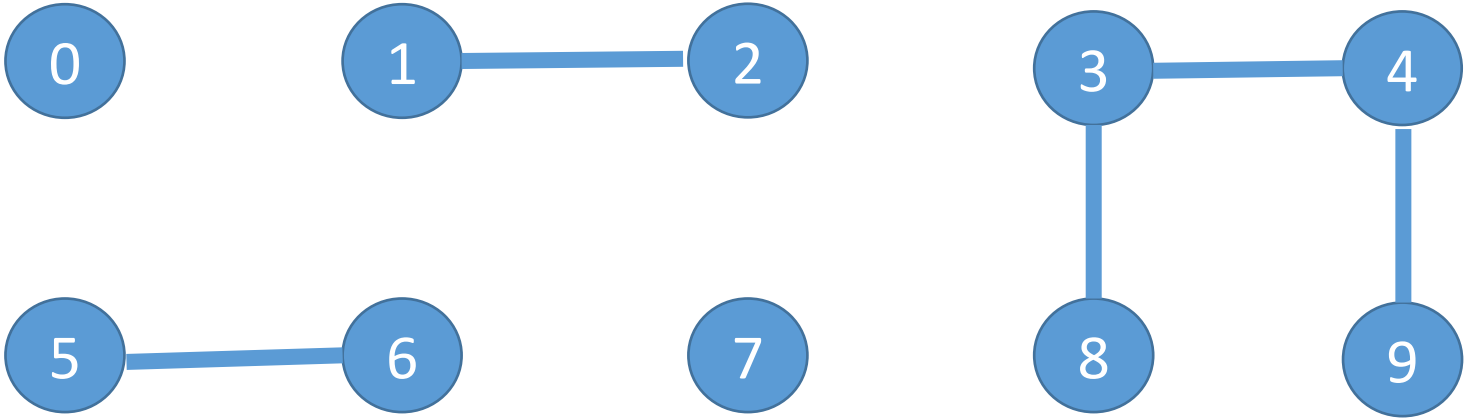
	0	1	2	3	4	5	6	7	8	9
cid[]	0	1	1	8	8	5	5	7	8	8



already connected

Quick demo

connected (5, 0)



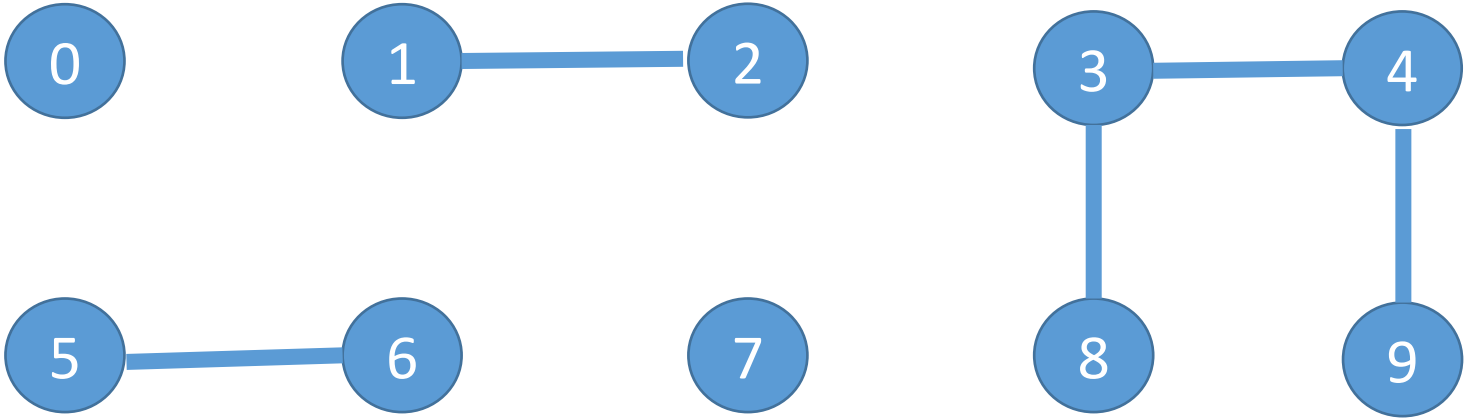
	0	1	2	3	4	5	6	7	8	9
cid[]	0	1	1	8	8	5	5	7	8	8



not connected

Quick demo

union (5, 0)

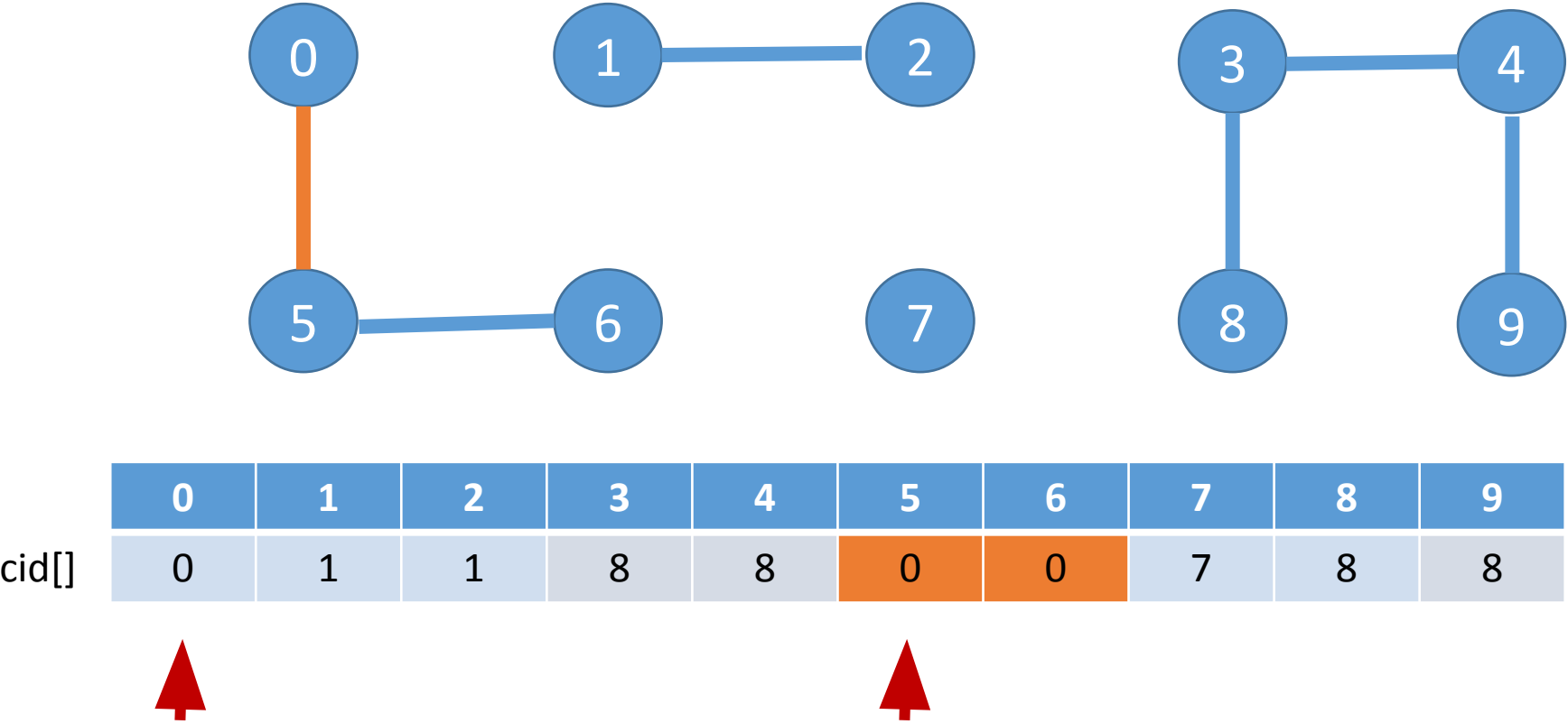


	0	1	2	3	4	5	6	7	8	9
cid[]	0	1	1	8	8	5	5	7	8	8



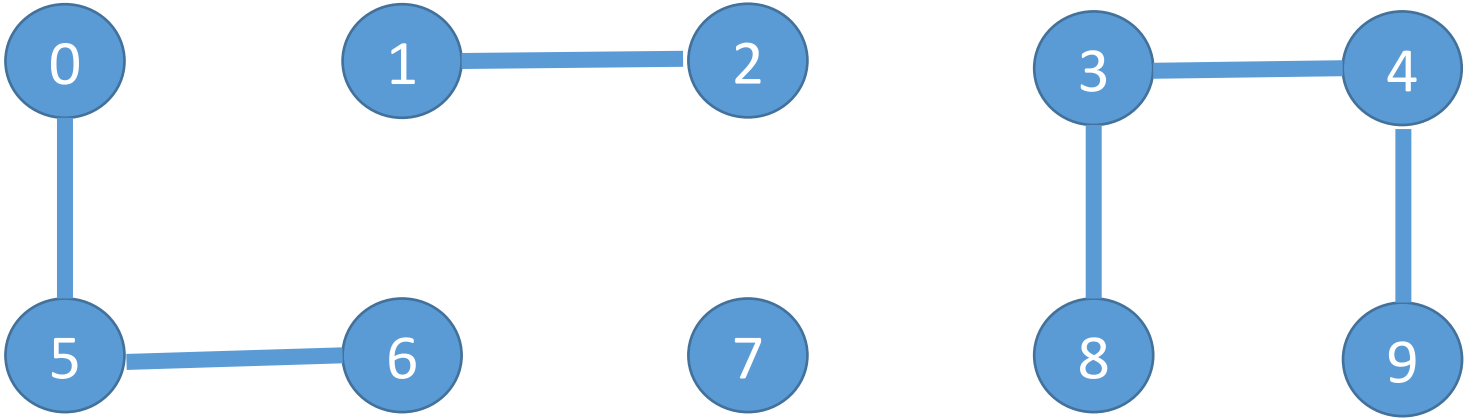
Quick demo

union (5, 0)



Quick demo

union (7, 2)

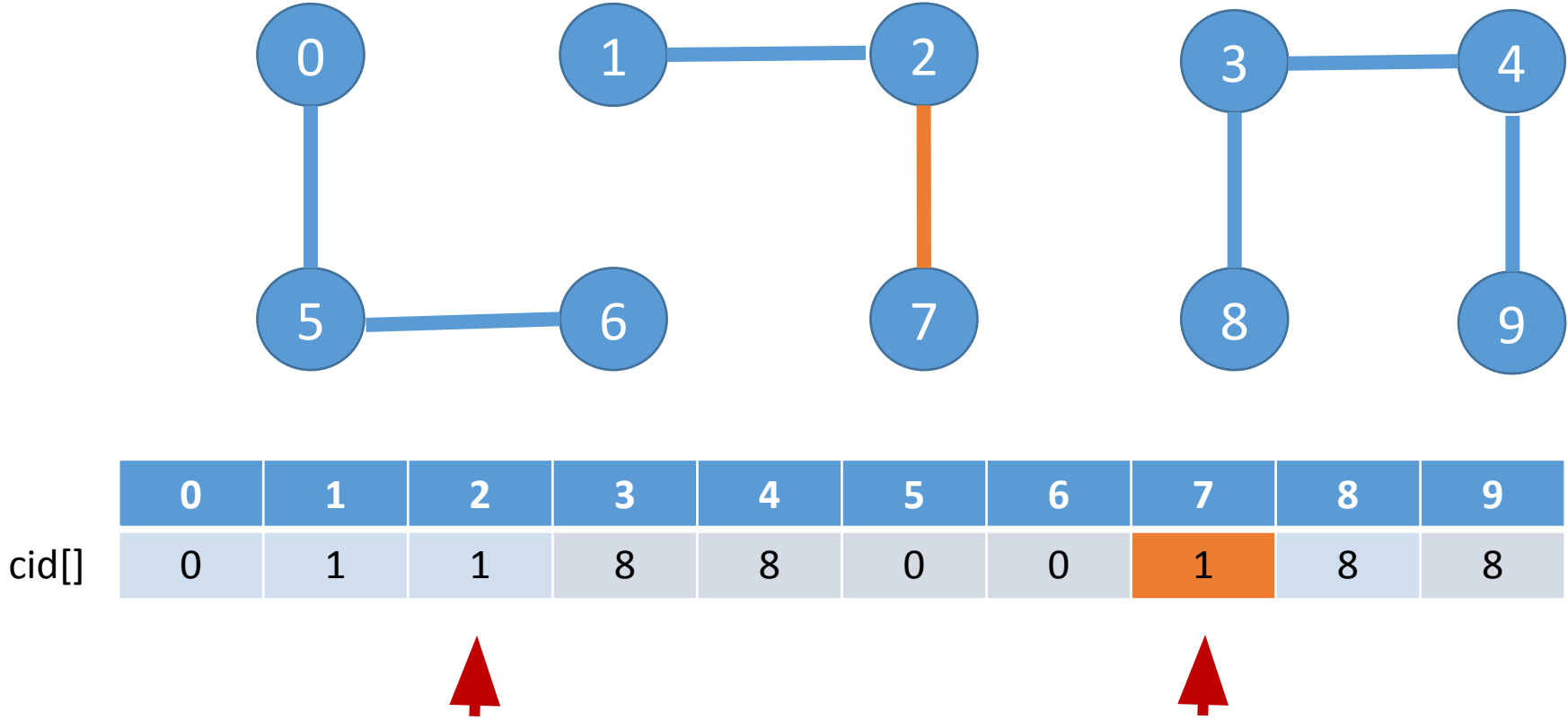


	0	1	2	3	4	5	6	7	8	9
cid[]	0	1	1	8	8	0	0	7	8	8



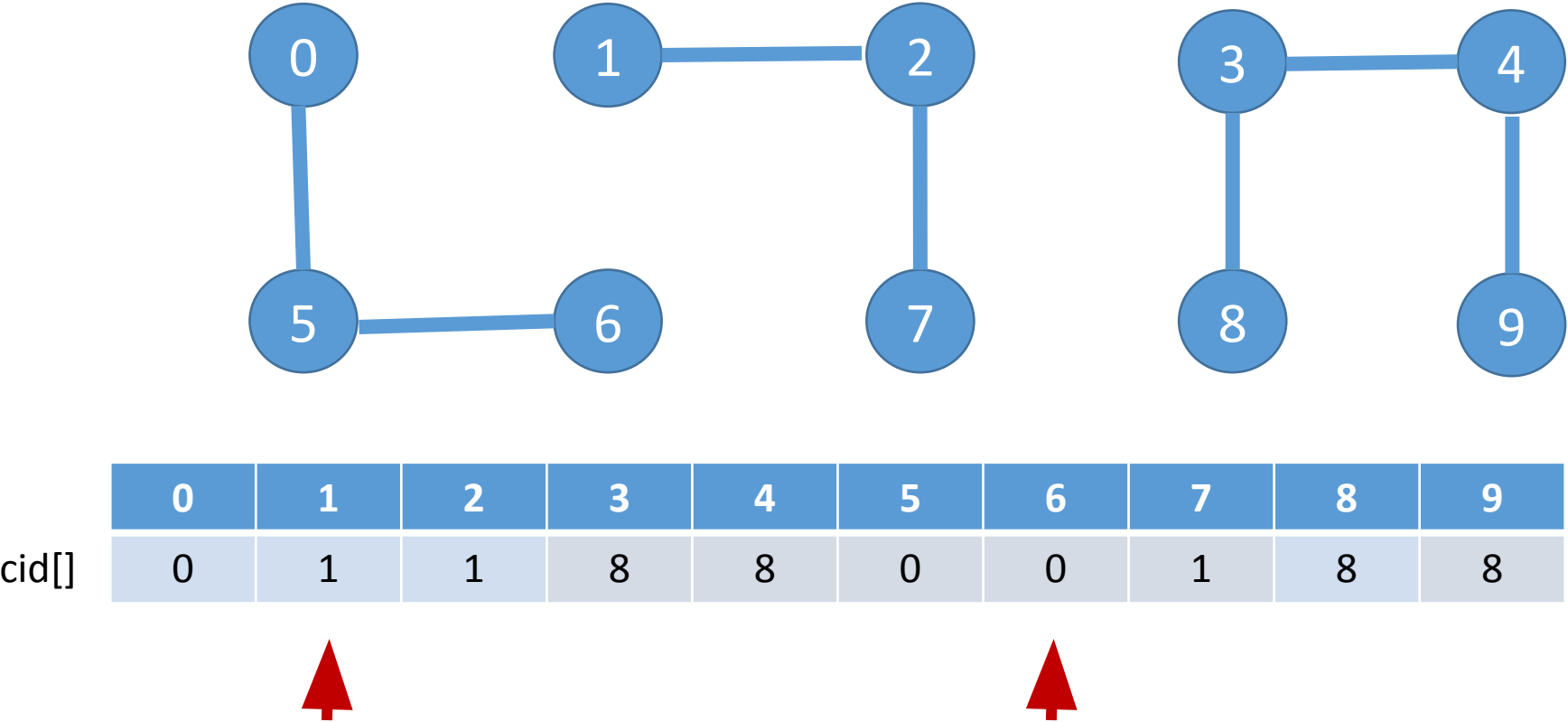
Quick demo

union (7, 2)



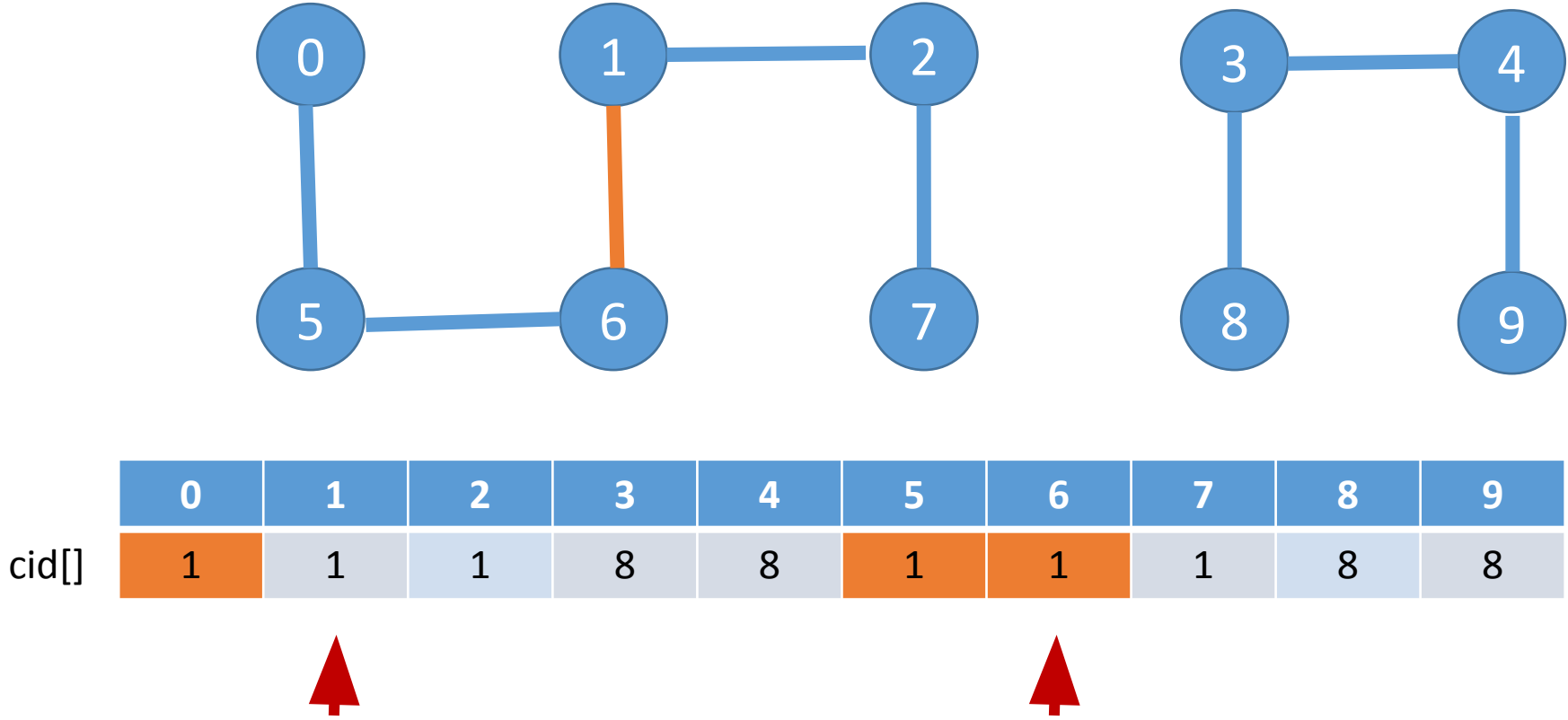
Quick demo

union (6, 1)

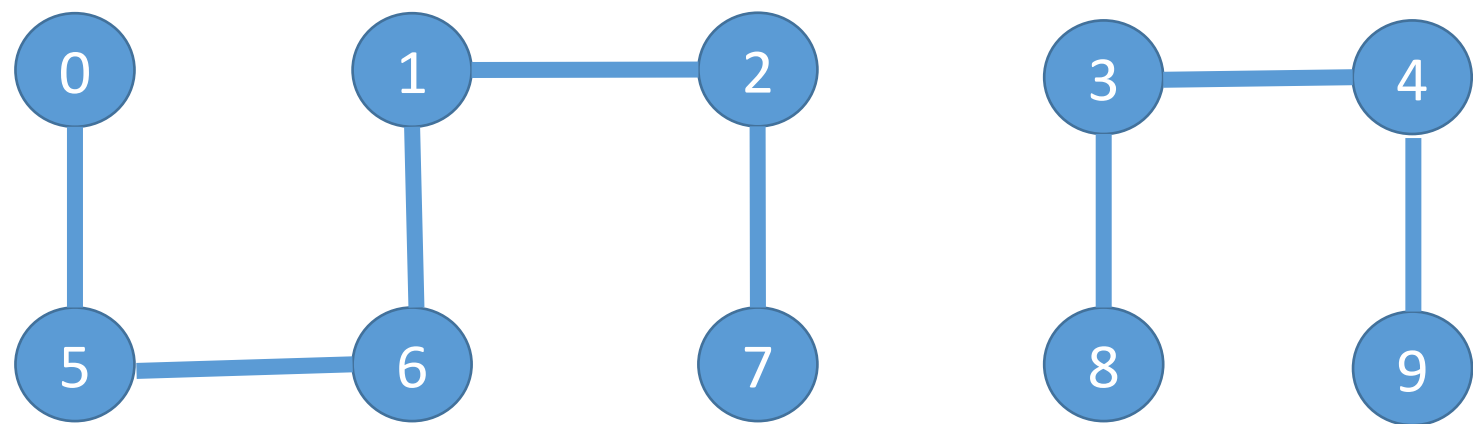


Quick demo

union (6, 1)



Quick demo



	0	1	2	3	4	5	6	7	8	9
cid[]	1	1	1	8	8	1	1	1	8	8

```
class QuickFindUF:
```

```
    cid = None
```

```
    size = 0
```

```
    def __init__(self, n):
```

```
        self.size = n
```

```
        self.cid = np.zeros(self.size)
```

```
        for i in range(self.size):
```

```
            self.cid[i] = i
```

```
    def connected(self, p, q)
```

```
        return self.cid[p] == self.cid[q]
```

```
    def union(self, p, q)
```

```
        if self.connected(p, q): return
```

```
        pid = self.cid[p]
```

```
        for i in range(self.size)
```

```
            if(self.cid[i] == pid): self.cid[i] = self.cid[q]
```

#set id of each object to itself
{n array access}

check whether p and q are in
the same component
(2 array accesses)

change all entries with id[p] to id[q]
(at most $2N + 2$ array accesses)

Cost Model: Number of array accesses (for read and write)

Algorithm	Initialize	Union	Find
Quick-find	N	N	1

Order of growth of number of array accesses

Quick find defect – Union too expensive

Ex: Takes N^2 array accesses to process sequence of N union commands on N objects.

Quadratic algorithms do not scale!!

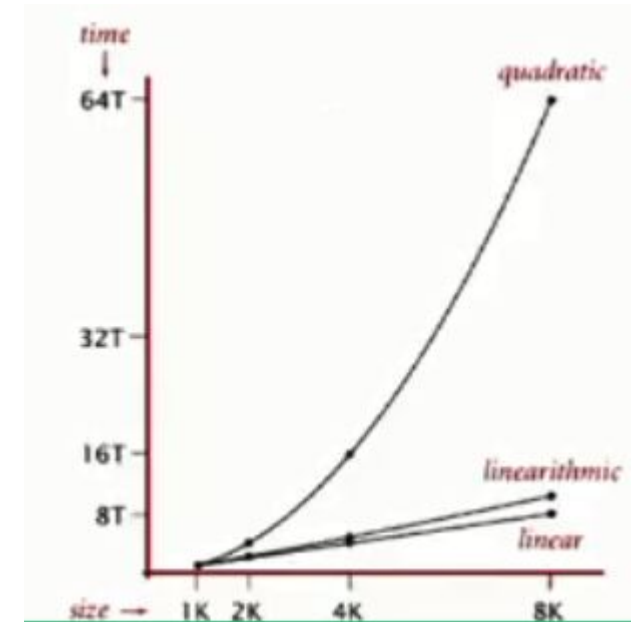
- Rough standard (for now).
 - 10^9 operation per second
 - 10^9 words of main memory
 - Touch all words in approximately 1 second
- Ex. Huge problem for quick-find
 - 10^9 union commands on 10^9 objects.
 - Quick-find takes more than 10^{18} operations.
 - 30+ years of computer time!!

Huge problem for quick find

- 10^9 union commands o 10^9 objects
- Quick-find takes more than 10^{18} operations
- 30+ years of computer time!!

Quadratic solutions don't scale with technology

- New computer may be 10X fast
- But, has 10X as much memory => want to solve a problem that is 10X as big
- With quadratic algorithm, takes 10X as long



QUIZ

What is the maximum number of **id[]** array entries that can change (from one value to a different value) during one call to *union* when using the quick-find data structure on **n** elements?

1. 1
2. $\log_2 n$
3. $n-1$
4. n

In the worst case, all of the entries except **id[q]** are changed from **id[p]** to **id[q]**.

1.5 UNION-FIND

- dynamic connectivity

- **quick find**

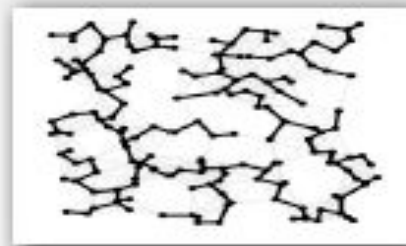
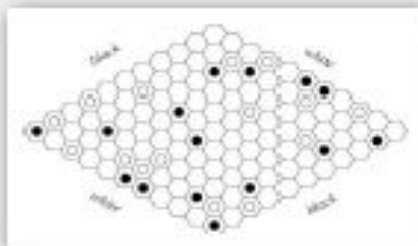
- quick union
- Improvements
- applications

1.5 UNION-FIND

- dynamic connectivity
- quick find
- quick union
- Improvements
- **Applications – OR assignment 1**

Union-find applications

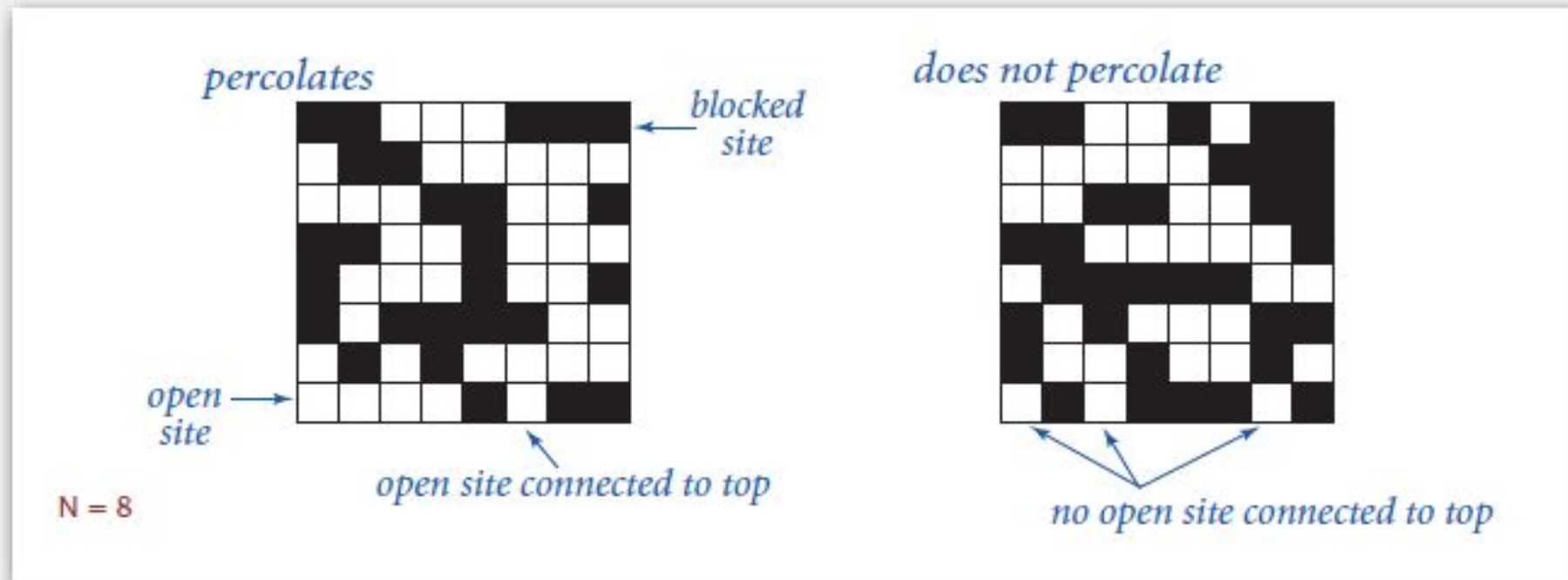
- Percolation.
- Games (Go, Hex).
- ✓ Dynamic connectivity.
 - Least common ancestor.
 - Equivalence of finite state automata.
 - Hoshen-Kopelman algorithm in physics.
 - Hinley-Milner polymorphic type inference.
 - Kruskal's minimum spanning tree algorithm.
 - Compiling equivalence statements in Fortran.
 - Morphological attribute openings and closings.
 - Matlab's `bwlabel()` function in image processing.



Percolation

A model for many physical systems:

- N -by- N grid of sites.
- Each site is open with probability p (or blocked with probability $1 - p$).
- System **percolates** iff top and bottom are connected by open sites.



Percolation

A model for many physical systems:

- N -by- N grid of sites.
- Each site is open with probability p (or blocked with probability $1 - p$).
- System **percolates** iff top and bottom are connected by open sites.

model	system	vacant site	occupied site	percolates
electricity	material	conductor	insulated	conducts
fluid flow	material	empty	blocked	porous
social interaction	population	person	empty	communicates

Likelihood of percolation

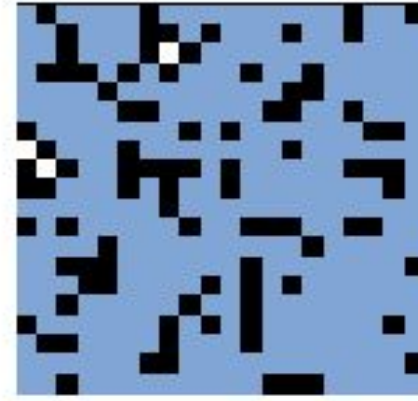
Depends on site vacancy probability p .



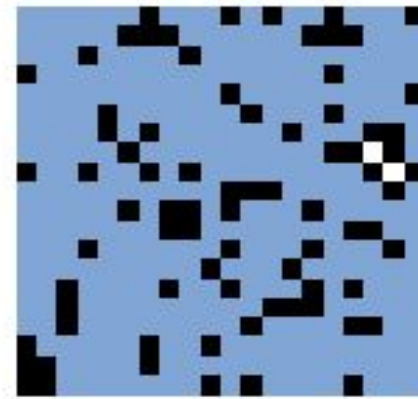
p low (0.4)
does not percolate



p medium (0.6)
percolates?



p high (0.8)
percolates

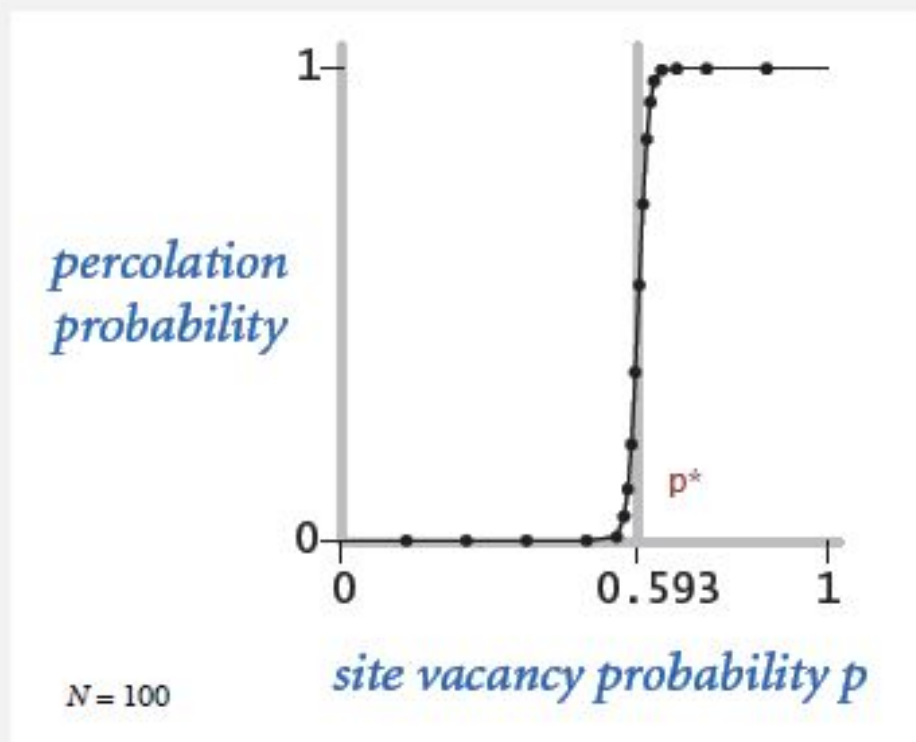


Percolation phase transition

When N is large, theory guarantees a sharp threshold p^* .

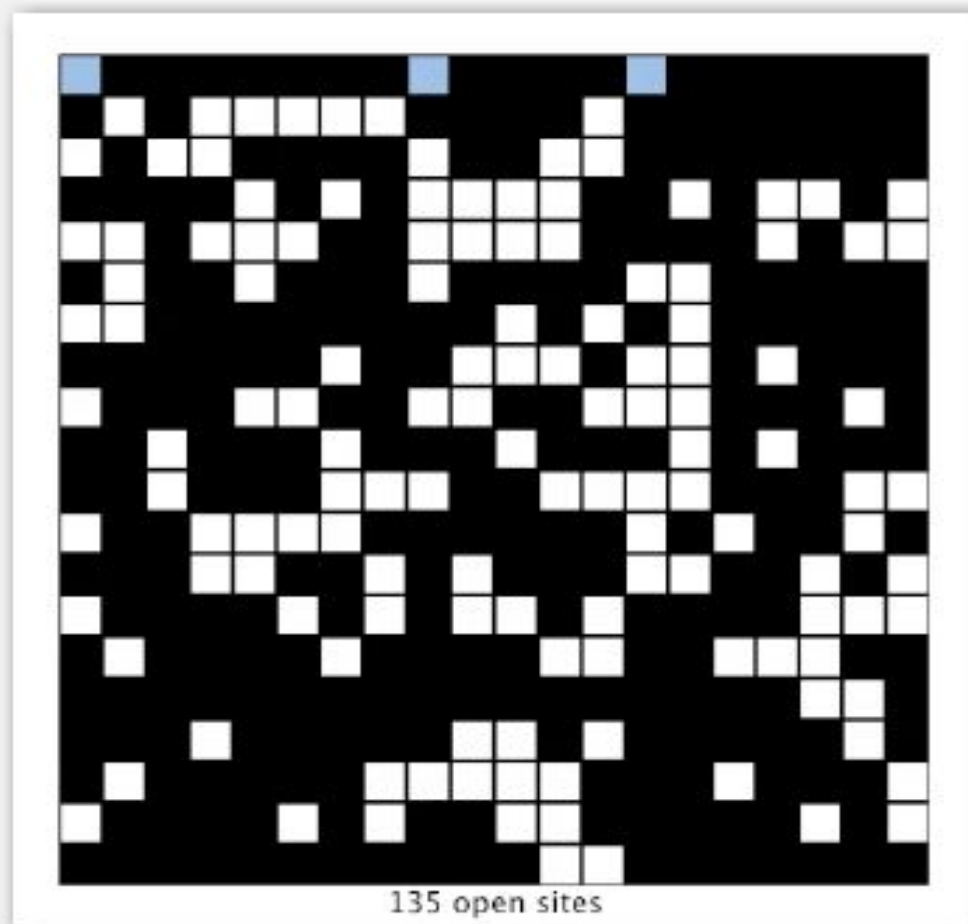
- $p > p^*$: almost certainly percolates.
- $p < p^*$: almost certainly does not percolate.

Q. What is the value of p^* ?



Monte Carlo simulation

- Initialize N -by- N whole grid to be blocked.
- Declare random sites open until top connected to bottom.
- Vacancy percentage estimates p^* .



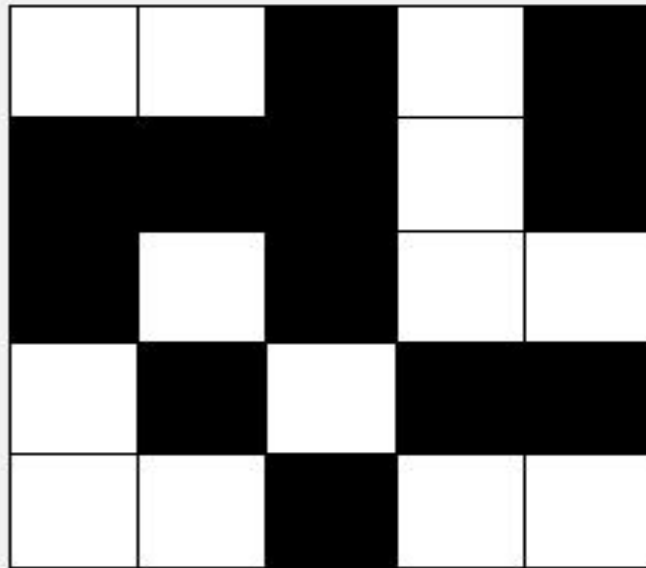
$N = 20$



Dynamic connectivity solution to estimate percolation threshold

Q. How to check whether an N -by- N system percolates?

$N = 5$



open site



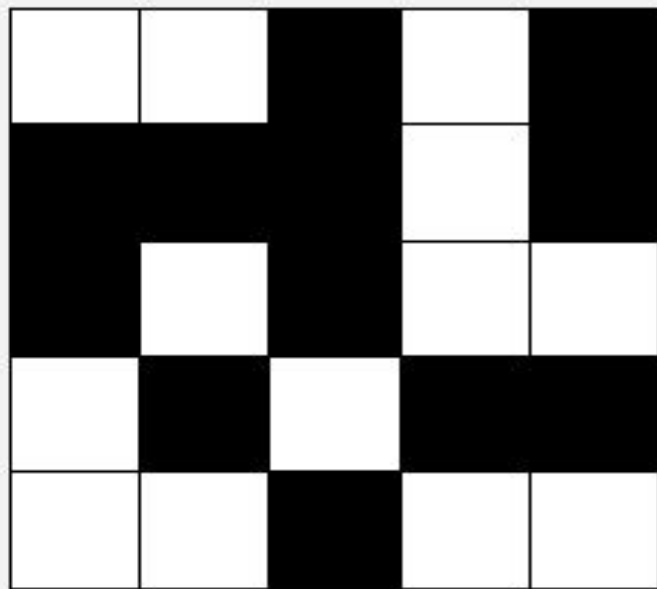
blocked site

Dynamic connectivity solution to estimate percolation threshold

Q. How to check whether an N -by- N system percolates?

- Create an object for each site and name them 0 to $N^2 - 1$.
- Sites are in same component if connected by open sites.

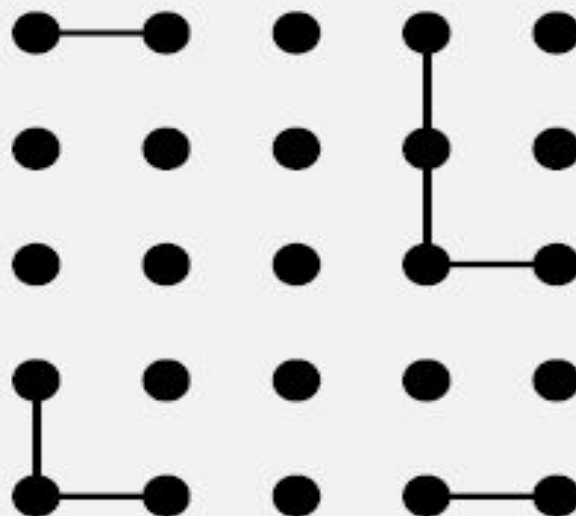
$N = 5$



open site



blocked site

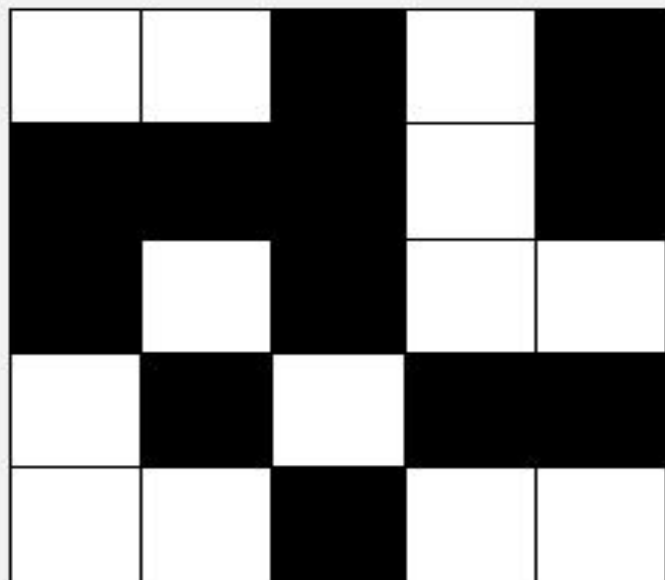


Dynamic connectivity solution to estimate percolation threshold

Q. How to check whether an N -by- N system percolates?

- Create an object for each site and name them 0 to $N^2 - 1$.

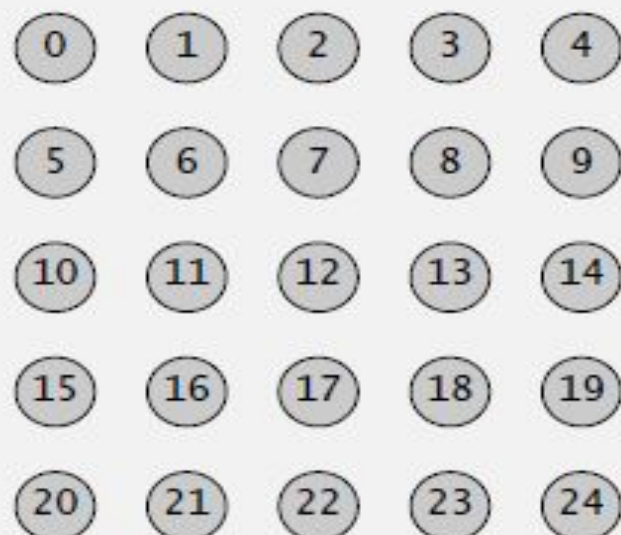
$N = 5$



open site



blocked site



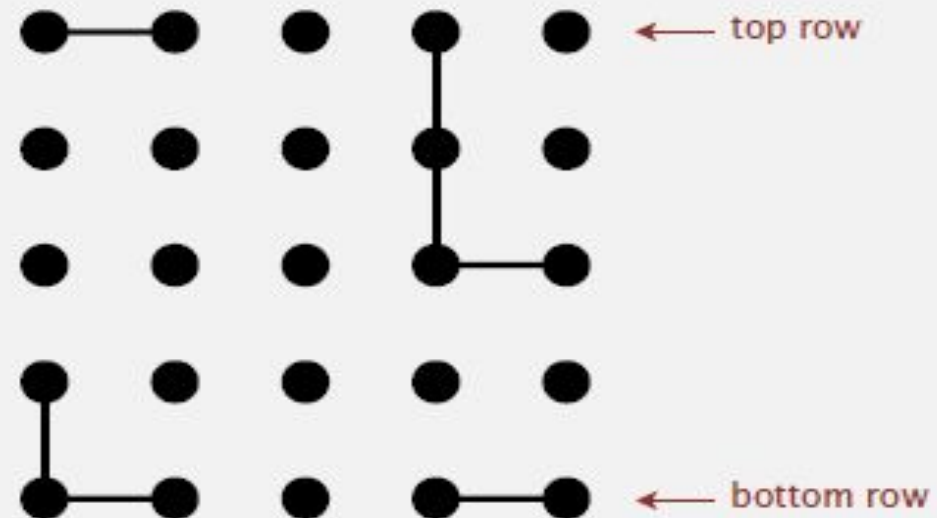
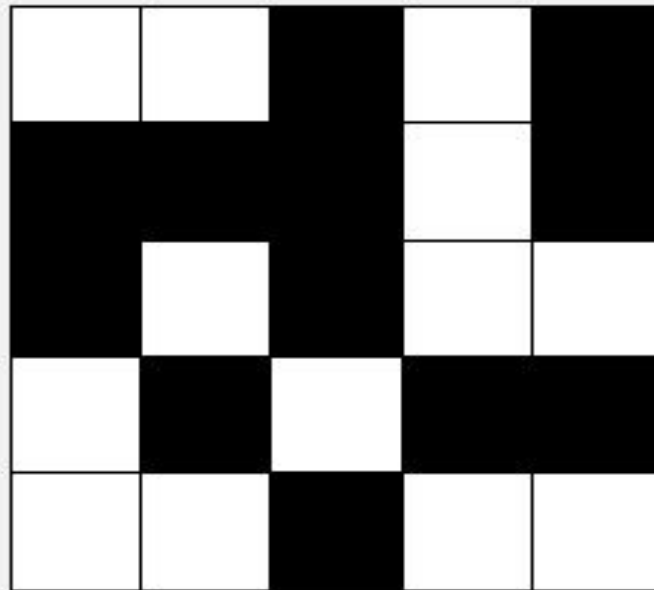
Dynamic connectivity solution to estimate percolation threshold

Q. How to check whether an N -by- N system percolates?

- Create an object for each site and name them 0 to $N^2 - 1$.
- Sites are in same component if connected by open sites.
- Percolates iff any site on bottom row is connected to site on top row.

brute-force algorithm: N^2 calls to `connected()`

$N = 5$



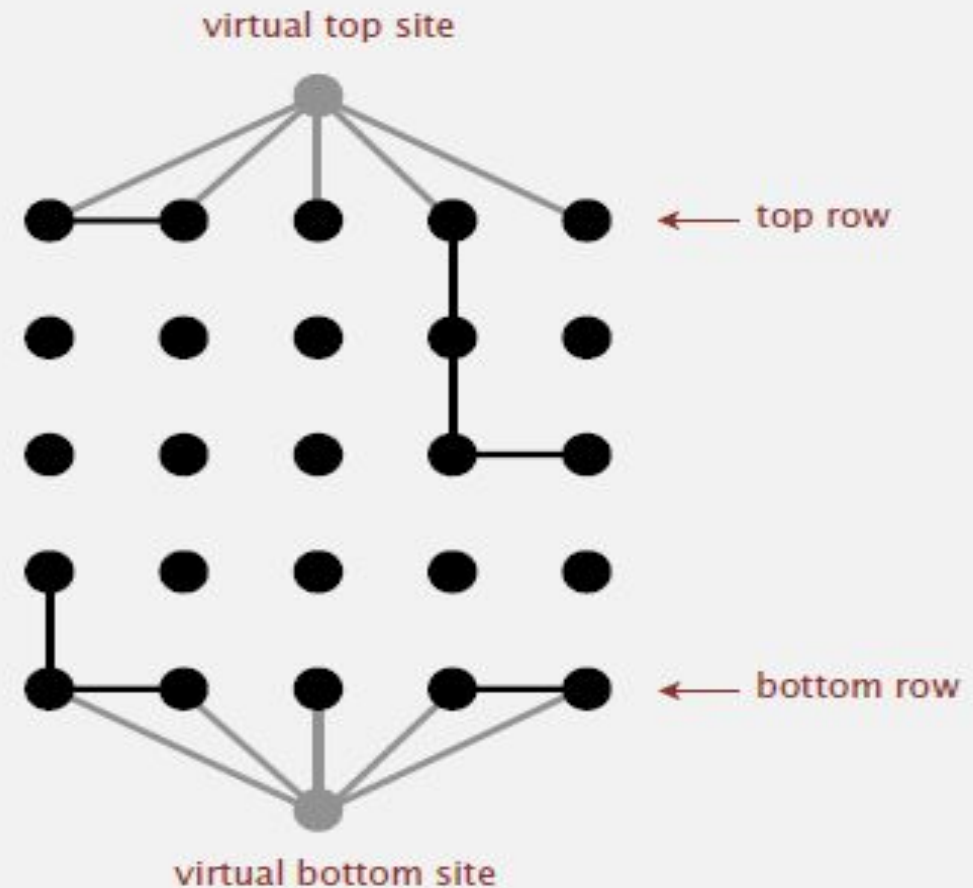
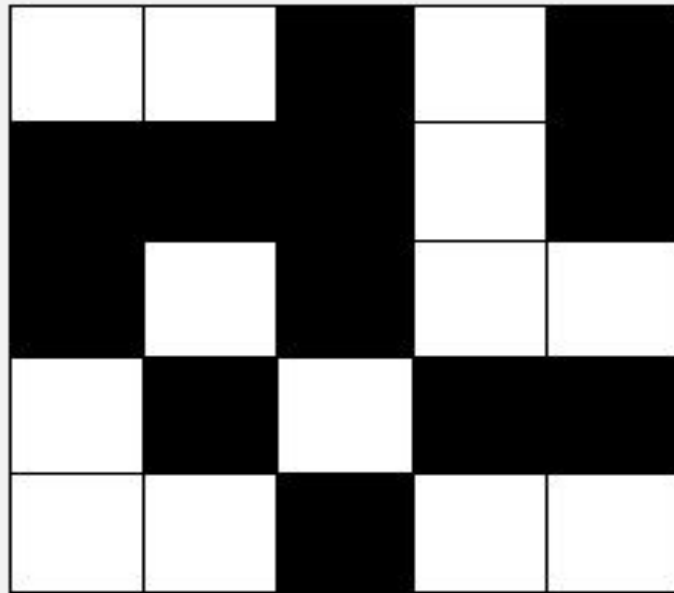
Dynamic connectivity solution to estimate percolation threshold

Clever trick. Introduce 2 virtual sites (and connections to top and bottom).

- Percolates iff virtual top site is connected to virtual bottom site.

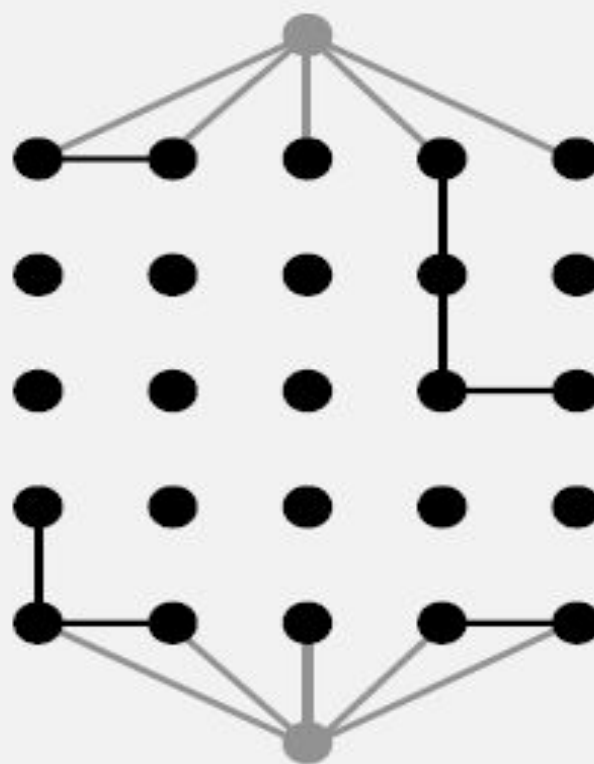
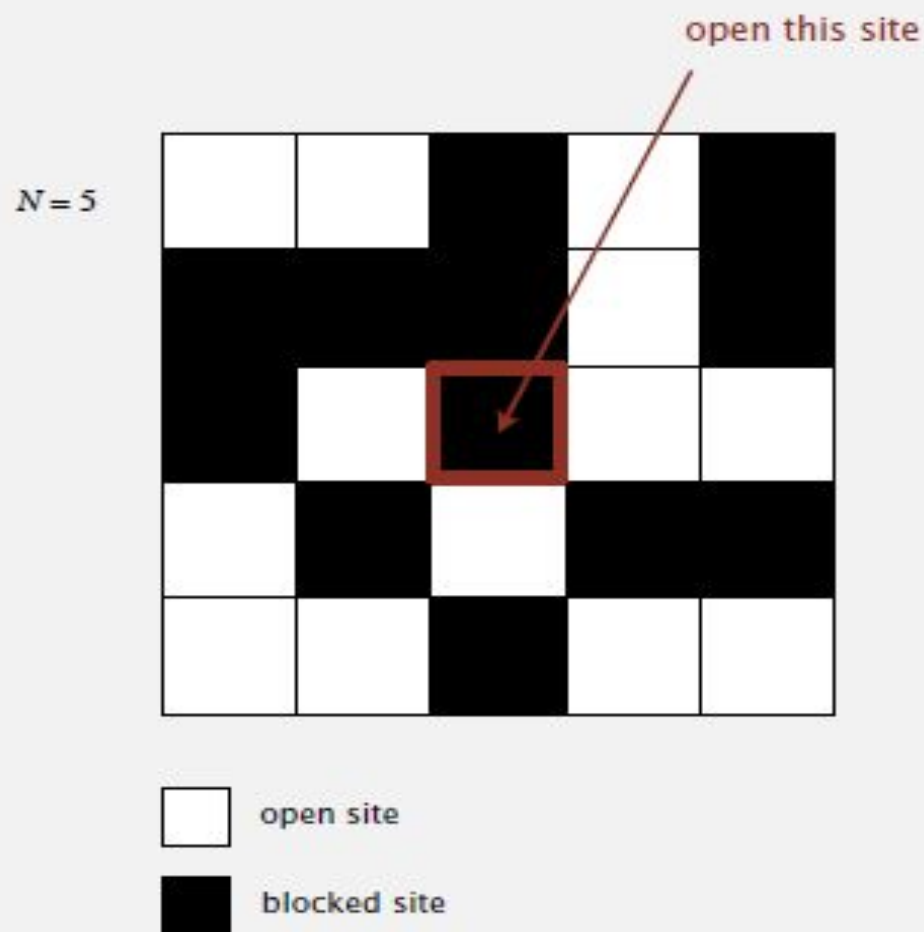
efficient algorithm: only 1 call to `connected()`

$N = 5$



Dynamic connectivity solution to estimate percolation threshold

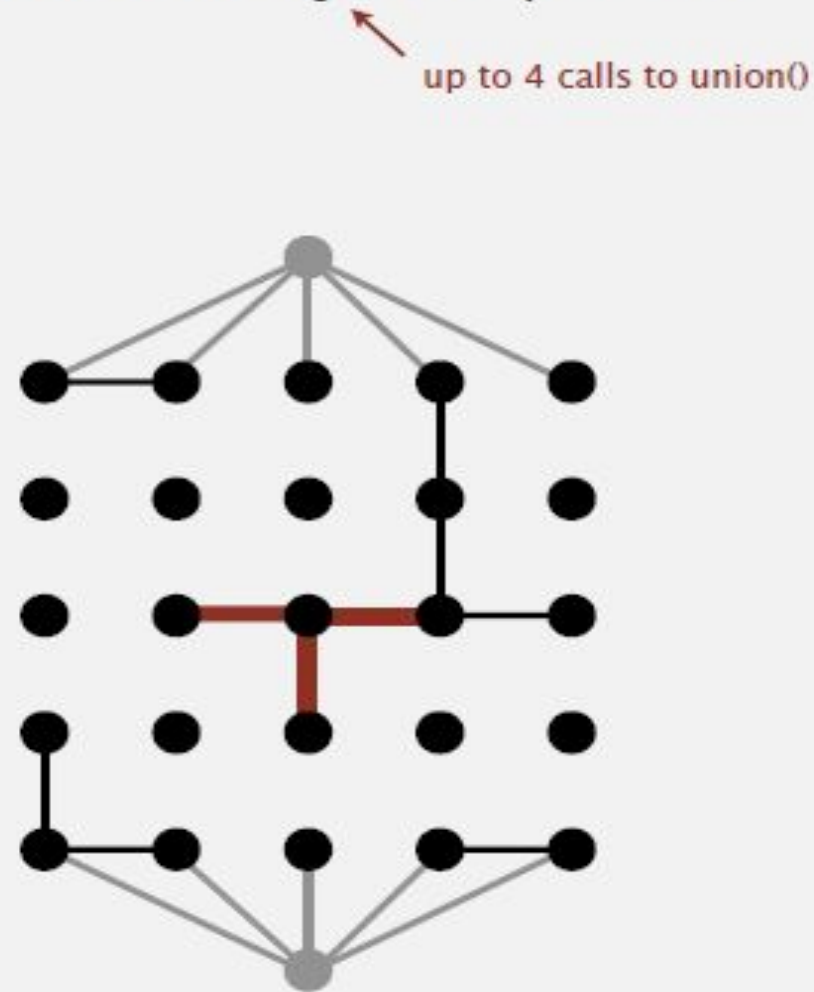
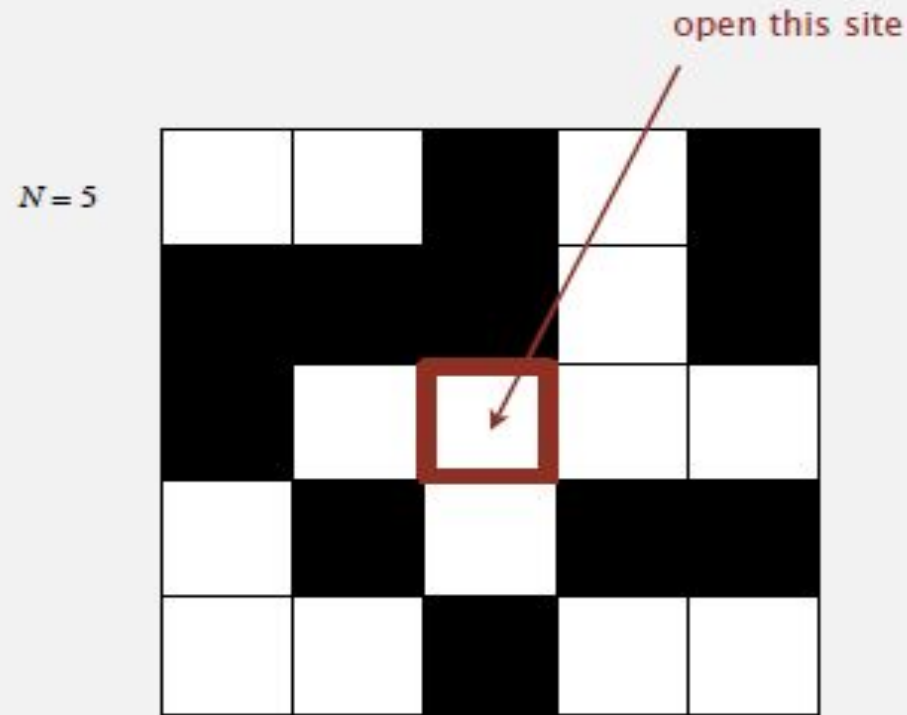
Q. How to model opening a new site?



Dynamic connectivity solution to estimate percolation threshold

Q. How to model opening a new site?

A. Mark new site as open; connect it to all of its adjacent open sites.

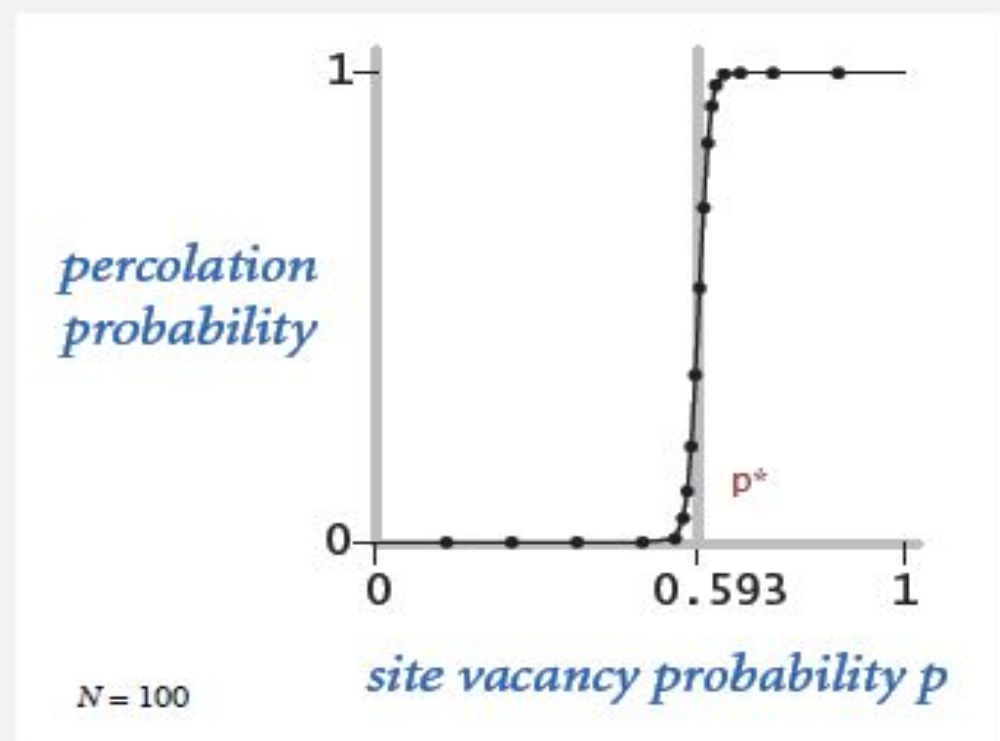


Percolation threshold

Q. What is percolation threshold p^* ?

A. About 0.592746 for large square lattices.

constant known only via simulation



Fast algorithm enables accurate answer to scientific question.

1.5 UNION-FIND

- dynamic connectivity
- quick find
- quick union
- Improvements
- **applications**

Subtext of today's lecture (and this course)

- Steps to developing a usable algorithm.
 - Model the problem.
 - Find an algorithm to solve it.
 - Fast enough? Fits in memory?
 - If not, figure out why.
 - Find a way to address the problem.
 - Iterate until satisfied.
- The scientific method.
- Mathematical analysis.

1.5 UNION-FIND

- dynamic connectivity
- quick find
- quick union
- Improvements
- applications

QUIZ

When opening one new site in the percolation simulation, how many times is ***union()*** called?

1. 0, 1, 2, 3 or 4
2. 1, 2, 3 or 4
3. 2, 3 or 4
4. 4

It is called for each neighbouring site that is already open. There are 4 possible neighbours, but some of them may not already be open.

Interview question 1:

Social network connectivity. Given a social network containing n members and a log file containing m timestamps at which times pairs of members formed friendships, design an algorithm to determine the earliest time at which all members are connected (i.e., every member is a friend of a friend of a friend ... of a friend). Assume that the log file is sorted by timestamp and that friendship is an equivalence relation. The running time of your algorithm should be $m \cdot \log n$ or better and use extra space proportional to n .

Union-find with specific canonical element. Add a method **find()** to the union-find data type so that **find(*i*)** returns the largest element in the connected component containing *i*. The operations, **union()**, **connected()**, and **find()** should all take logarithmic time or better.

For example, if one of the connected components is **{1,2,6,9}**, then the **find()** method should return **9** for each of the four elements in the connected components.

Successor with delete. Given a set of n integers $S = \{0, 1, \dots, n-1\}$ and a sequence of requests of the following form:

- Remove x from S
- Find the *successor* of x : the smallest y in S such that $y \geq x$.

design a data type so that all operations (except construction) take logarithmic time or better in the worst case.